

Visión Artificial

Tema 5. Redes neuronales convolucionales para visión por computador.

PEDRO ENRIQUE LÓPEZ DE TERUEL ALCOLEA
DEPARTAMENTO INGENIERÍA Y TECNOLOGÍA DE COMPUTADORES
FACULTAD DE INFORMÁTICA

Universidad de Murcia

Máster en Inteligencia Artificial

2025/2026



Redes neuronales para visión artificial

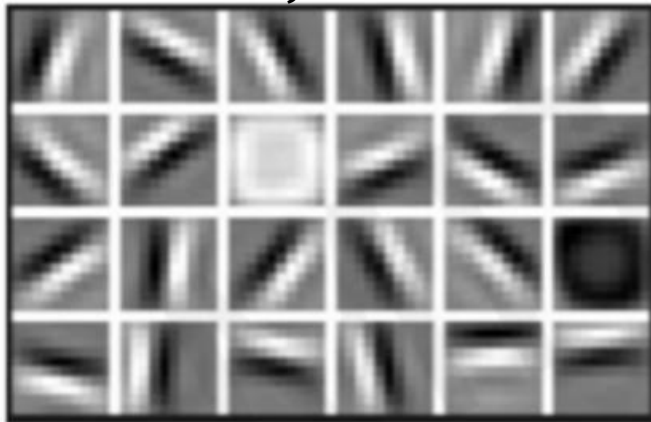
Objetivos del tema



- ▶ Extracción automatizada de características:
 - Alternativa a la extracción de *features* “*hand-engineered*”: aprender a partir de ejemplos.
 - Comprender el ciclo básico de entrenamiento (*training, offline*) y uso (*inference, online*) de soluciones de visión artificial (VA) basadas en **Deep Learning** (DL).
- ▶ Computación diferenciable:
 - Comprender los fundamentos de la “computación diferenciable” (*backpropagation algorithm*) que permiten todo el enfoque.
- ▶ Software y arquitecturas DL en VA:
 - Introducción a *frameworks* de desarrollo de modelos de aprendizaje profundo, como **Keras+Tensorflow, pytorch**.
 - Estudio de principales **arquitecturas neuronales** DL para clasificación, localización y regresión en VA.

- Alternativa a las *hand-engineered features*:
 - ¿Podemos aprender las *features* subyacentes directamente a partir de los datos, basándonos directamente en decirle a nuestro modelo de computación qué salida(s) esperamos de él en cada ejemplo proporcionado para el aprendizaje?

**Features de
bajo nivel**



Bordes, *blobs*...

**Features de nivel
intermedio**



Ojos, nariz, boca...

**Features de
alto nivel**



Estructura facial

Redes neuronales para visión artificial

Resurgimiento de las NNs



- ▶ Todo el enfoque DL está basado en los (antiguos) modelos conexionistas: redes neuronales, o *neural networks* (NN).
- ▶ En realidad las bases se establecieron hace años, pero tres factores han provocado el reciente *boom*:

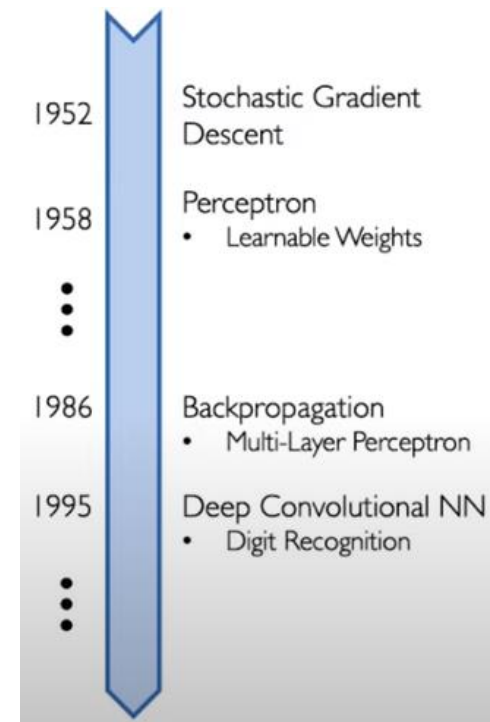
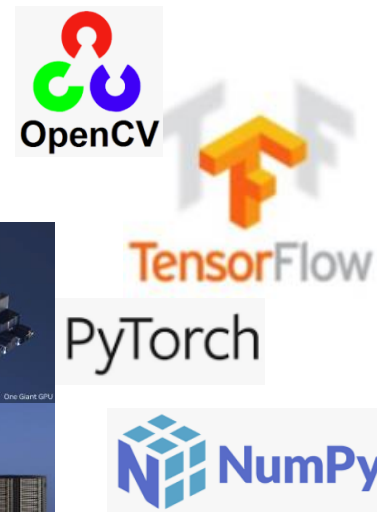
Big Data



Potencia hardware



Software abierto

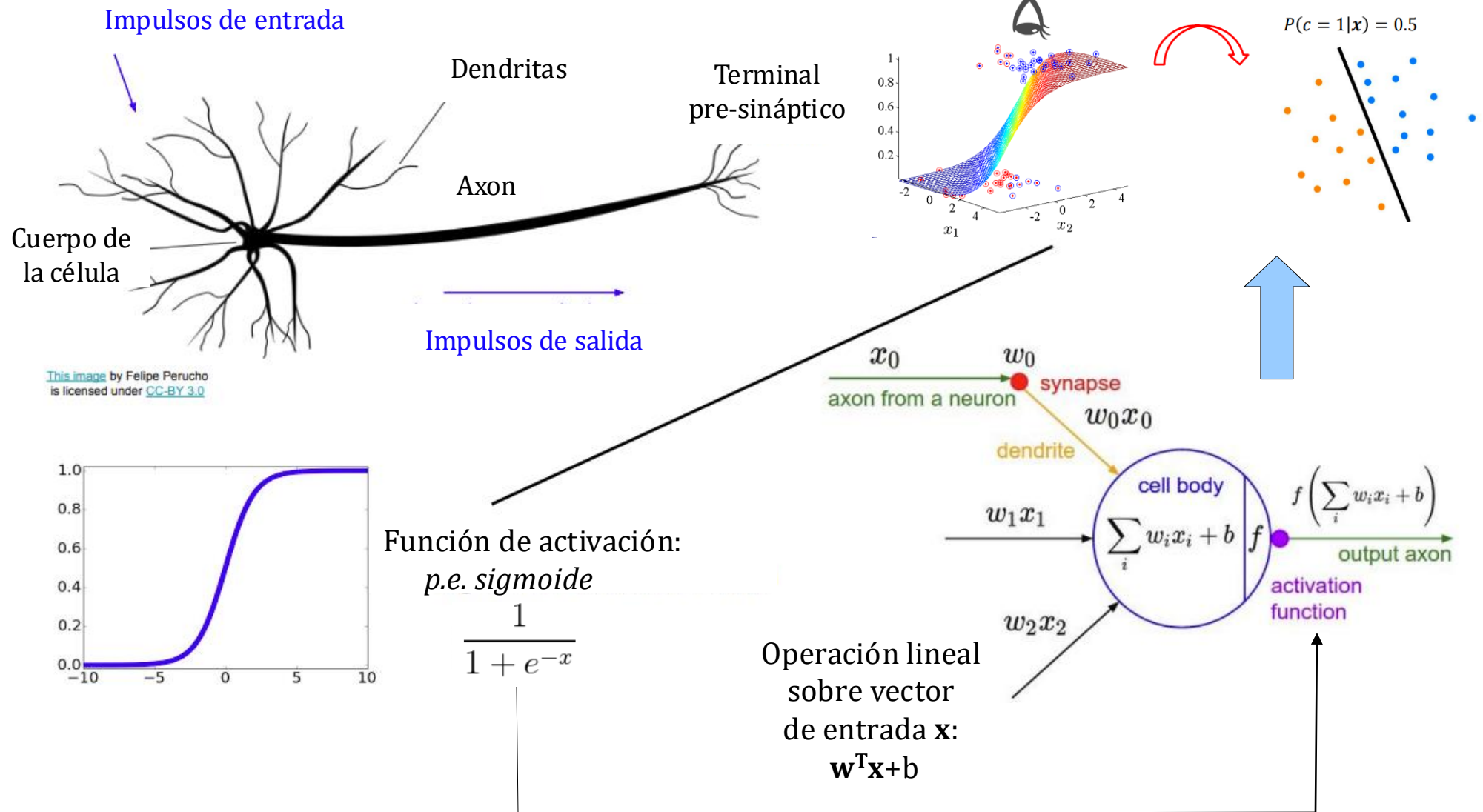


Redes neuronales para visión artificial

Neurona computacional

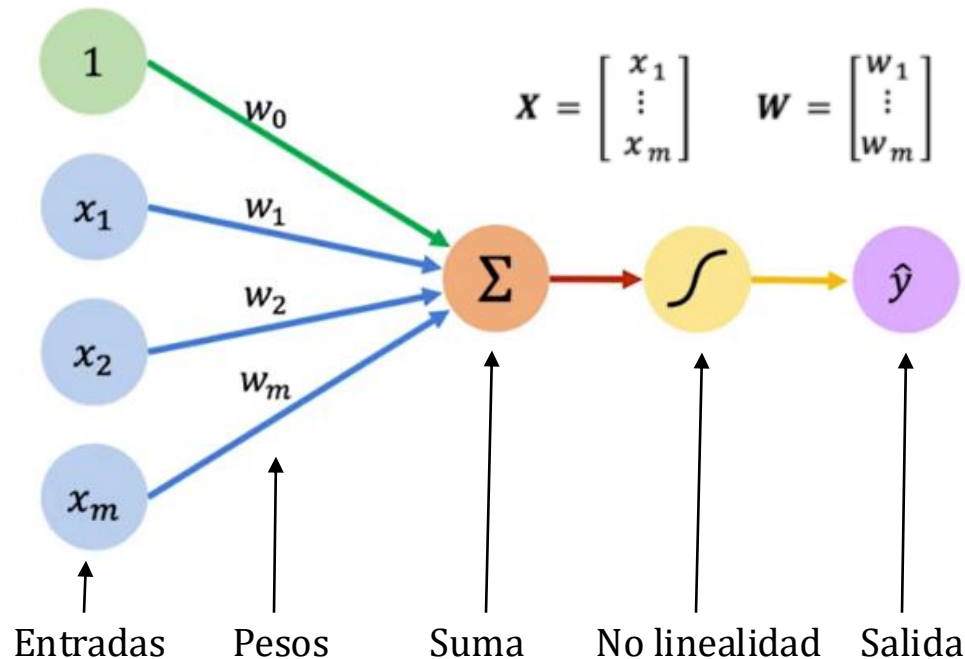


► Analogía con neurona biológica:



Redes neuronales para visión artificial

Bloque constructivo: *perceptron*

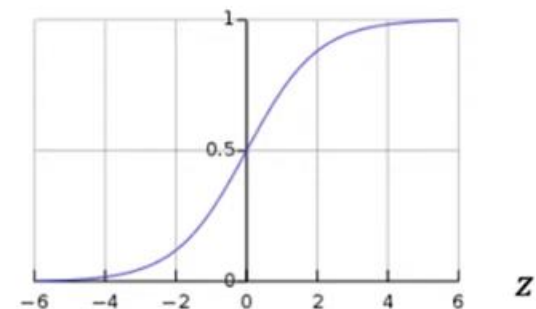


Función de activación

$$\hat{y} = g(w_0 + X^T W)$$

Por ejemplo, una sigmoide:

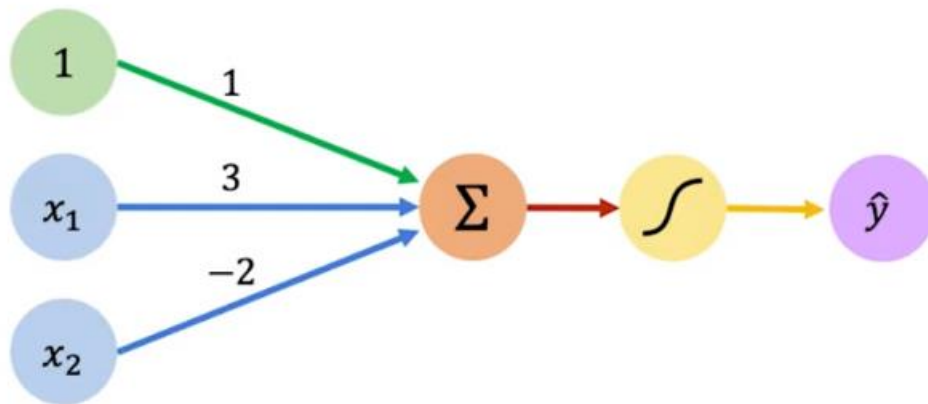
$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



- La sigmoide fue la primera, pero en realidad ya ni siquiera es la función de activación más utilizada.
 - Motivo: plantea ciertos problemas en redes profundas.
 - En transparencias posteriores veremos posibles alternativas.

Redes neuronales para visión artificial

Bloque constructivo: *perceptron*



Suponer una entrada: $\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

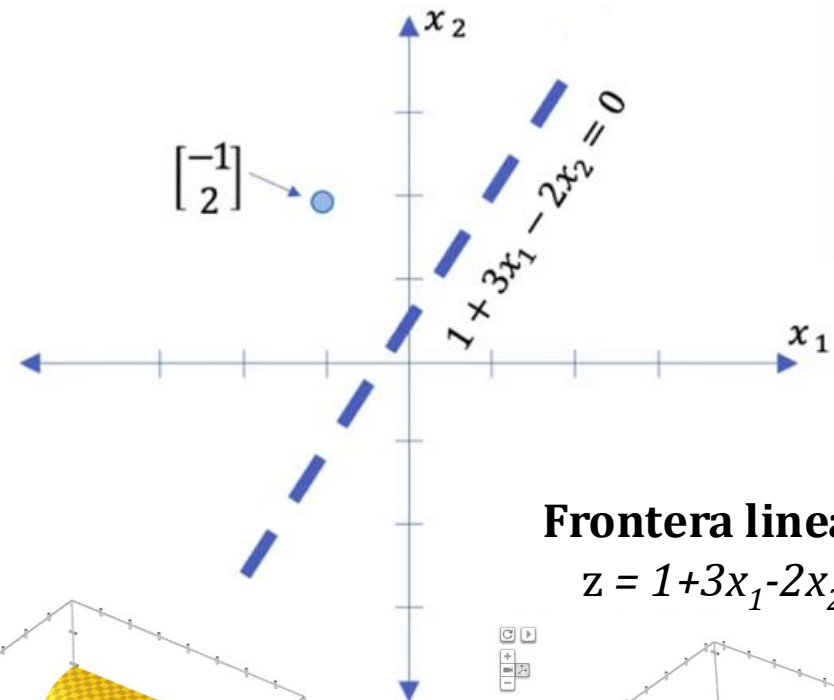
$$\begin{aligned}\hat{y} &= g(1 + (3 * -1) - (2 * 2)) \\ &= g(-6) \approx 0.002\end{aligned}$$

Con no linealidad:

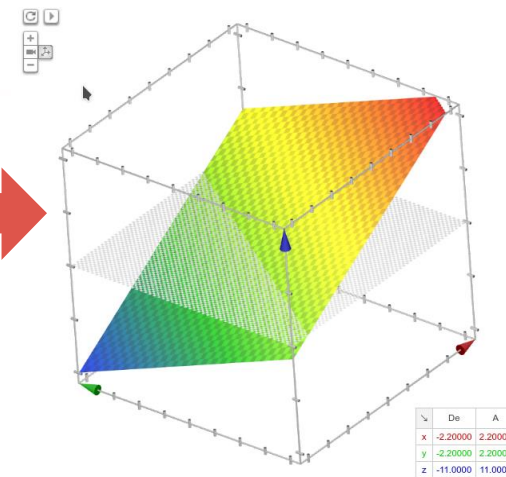
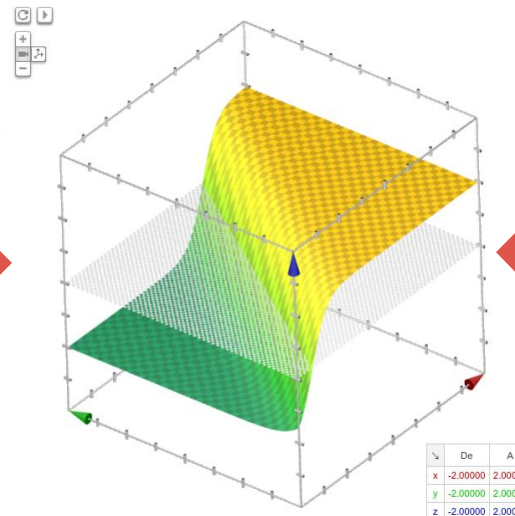
$$g(z) := \tanh(z)$$

$$\hat{y} = \tanh(1 + 3x_1 - 2x_2)$$

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



Frontera lineal:
 $z = 1 + 3x_1 - 2x_2$

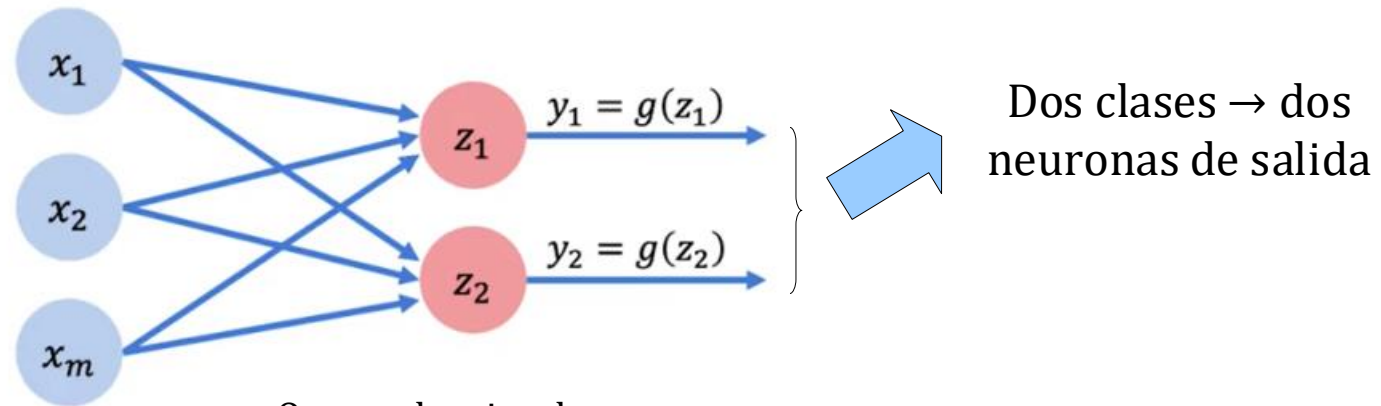


Redes neuronales para visión artificial

Perceptron multisalida



- Todas las entradas están conectadas a todas las salidas → Se denomina **capa densa**:



Que no despiste la notación distinta para b

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Ahora los coeficientes son una matriz W , los *bias* un vector b , y la salida también un vector:

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b} \quad \mathbf{y} = g(\mathbf{z})$$

```
import tensorflow as tf  
  
layer = tf.keras.layers.Dense(  
    units=2)
```

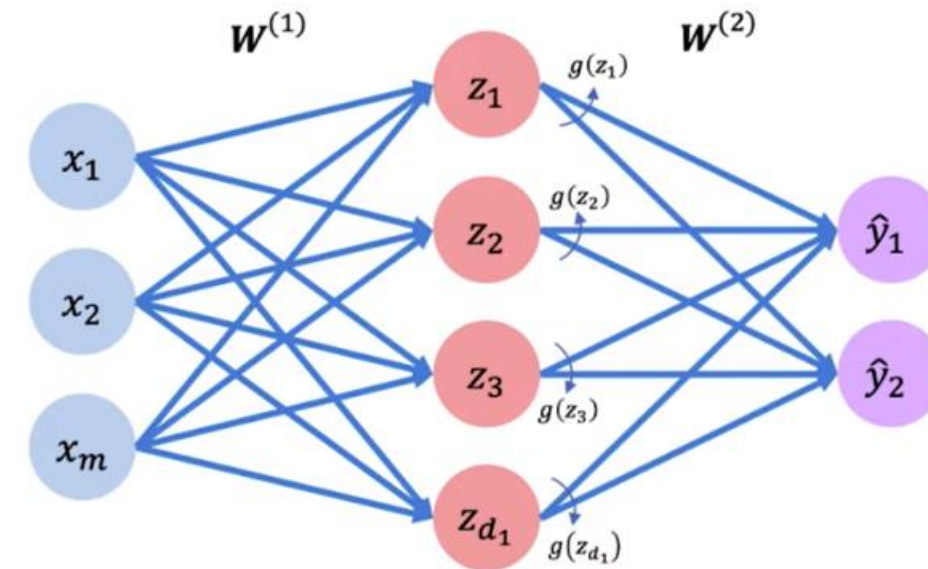
Ilustración para tres clases: <http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/>

Redes neuronales para visión artificial

NN con una capa oculta



- Una **capa densa** oculta + una **capa de salida**:



Cada capa:
*mapeo no lineal
entre espacios
vectoriales*

Entradas

Capa oculta

Salida final

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

Visualización de deformación del espacio de entrada para hacerlo linealmente separable:

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

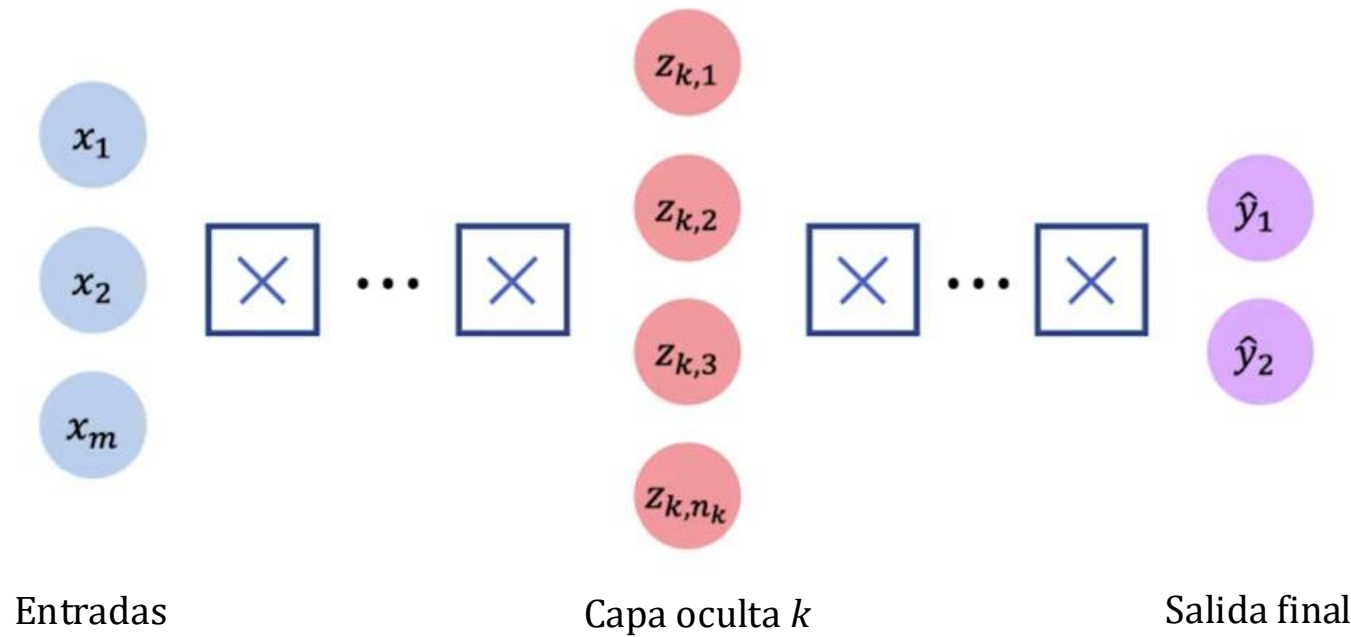
Redes neuronales para visión artificial

Deep Neural Network (DNN)



Máster en
Inteligencia
Artificial
UMU

- Una secuencia de **capas densas** ocultas + una **capa de salida**:



```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n1),
    tf.keras.layers.Dense(n2),
    ...
    tf.keras.layers.Dense(2)
])
```

$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

Sencilla red neuronal multicapa (interactiva y configurable) en espacio de entrada 2D:

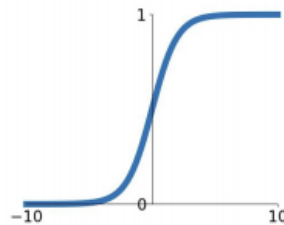
<https://playground.tensorflow.org/>

► Muchas posibilidades (pero siempre hace falta una **no linealidad**^(*)):

- Diferentes propiedades durante el aprendizaje (convergencia durante la optimización) y la inferencia (eficiencia durante la ejecución):

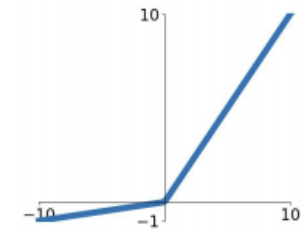
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



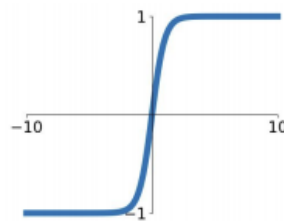
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

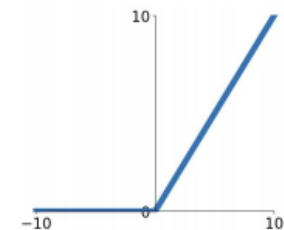


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

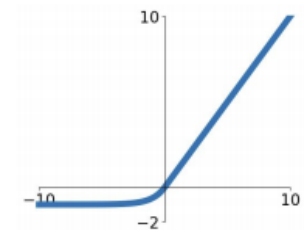
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



(*)En otro caso, un perceptrón multicapa sería equivalente a uno monocapa (multiplicar vector por matrices, en serie -> efecto equivalente a multiplicar por matriz resultado de la multiplicación de la secuencia de matrices).

Redes neuronales para visión artificial

Otras posibles funciones de activación (con derivadas)



Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

Y muchas más: *swish function*, *hard swish*, etc...

► No linealidad vectorial:

- Utilizada para NNs multiclase:



Queremos interpretar las salidas de la red como probabilidades

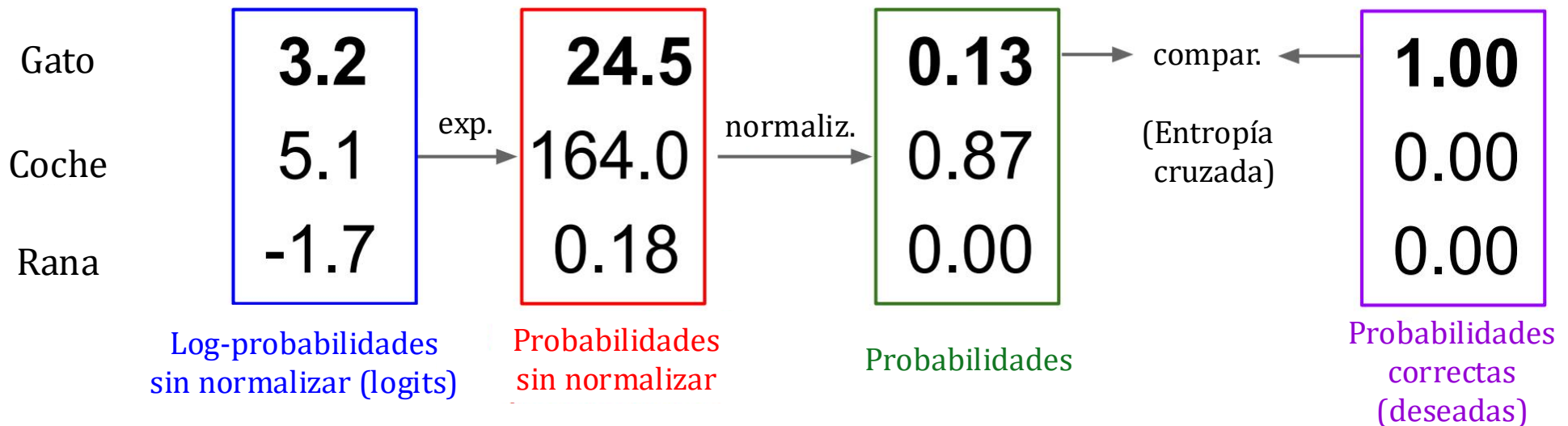
$$s = f(x_i; W)$$

Probabilidades
han de ser ≥ 0

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Función softmax

Probabilidades
deben sumar 1



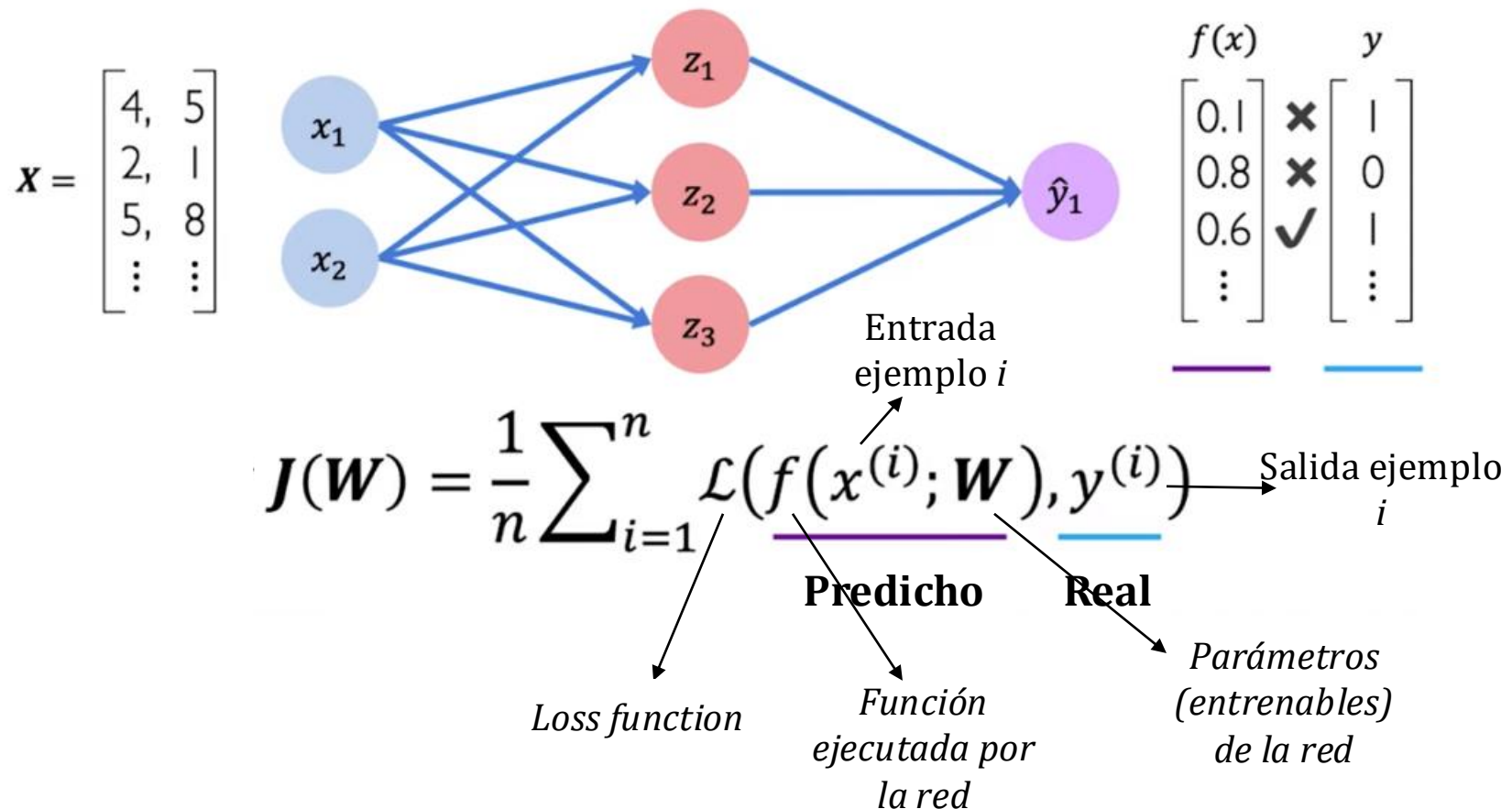
Fei-Fei Li & Justin Johnson & Serena Yeung: [enlace](#)

Redes neuronales para visión artificial

Función objetivo (ejemplo clasificación binaria 0/1)



- La **función de costo** (*loss function*, o *objective function*) mide el costo en el que se incurre al realizar predicciones incorrectas:

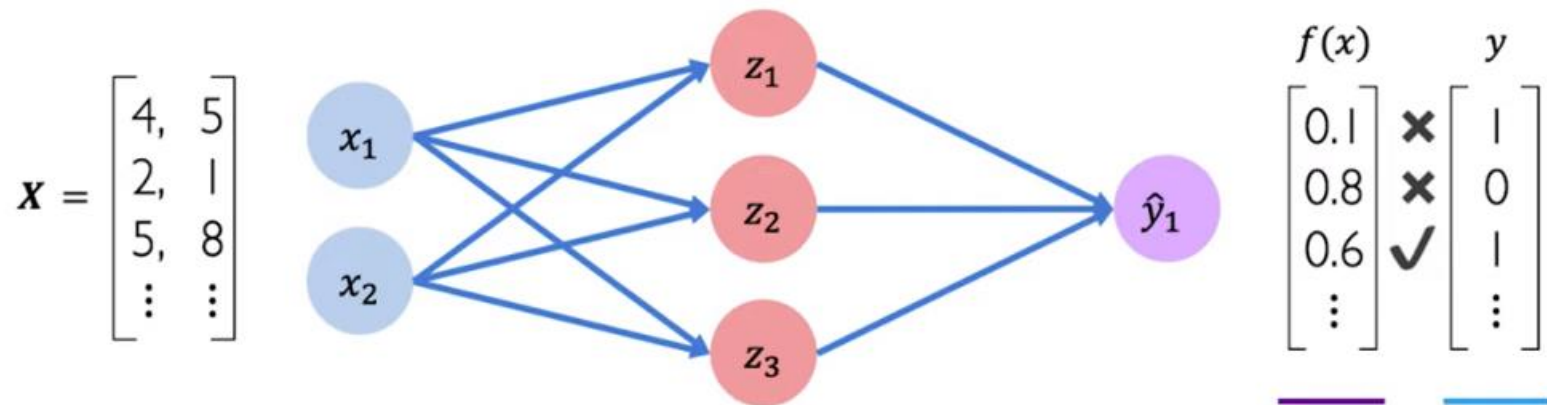


Redes neuronales para visión artificial

Ejemplo concreto *loss function*: entropía cruzada binaria



- Función objetivo (***binary cross entropy (BCE) loss***) que se suele utilizar cuando el modelo produce una salida probabilística (entre 0.0 y 1.0):



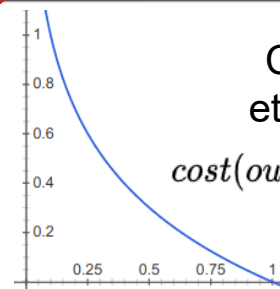
$$J(\mathbf{W}) = -\frac{1}{n} \sum_{i=1}^n \underbrace{y^{(i)}}_{\text{Real}} \log \left(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicho}} \right) + (1 - \underbrace{y^{(i)}}_{\text{Real}}) \log \left(1 - \underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicho}} \right)$$

```
loss = tf.reduce_mean( tf.nn.softmax_cross_entropy_with_logits(y, predicted) )
```

En realidad, simple caso particular de BCE multiclase (más general):

$$L(\hat{y}, y) = - \sum_k^K y^{(k)} \log \hat{y}^{(k)}$$

"One-hot encoding"

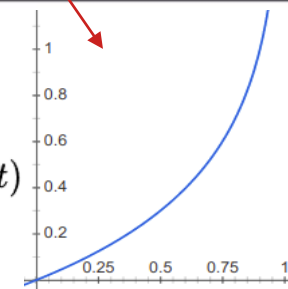


Coste si
etiqueta=1

$$\text{cost}(\text{out}) = -\log(\text{out})$$

Coste si
etiqueta=0

$$\text{cost}(\text{out}) = -\log(1 - \text{out})$$

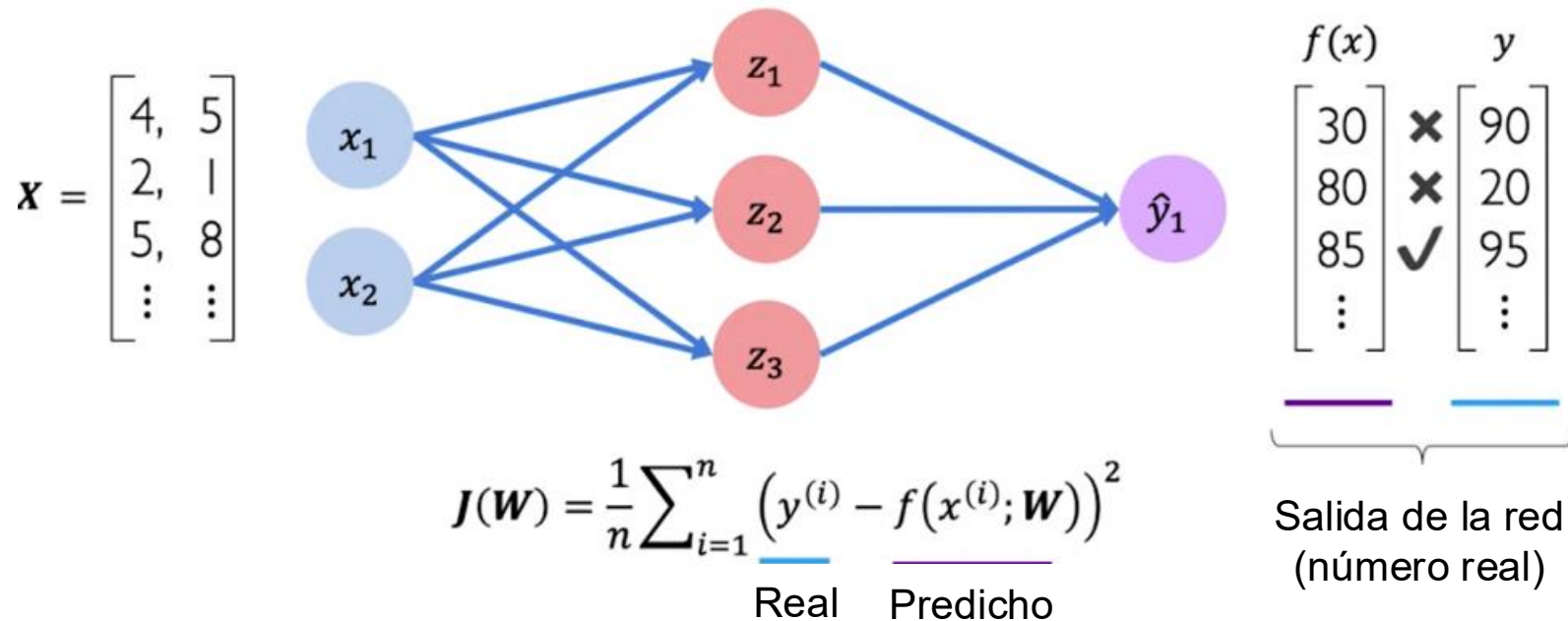


Redes neuronales para visión artificial

Ejemplo para regresión: MSE



- Para NNs cuyas salidas son valores reales continuos (*regresoras*) se suele usar el error cuadrático medio (***Mean Square Error loss***) como función objetivo:



```
loss = tf.reduce_mean( tf.square(tf.subtract(y, predicted)) )  
loss = tf.keras.losses.MSE( y, predicted )
```

Redes neuronales para visión artificial

Entrenamiento de NNs



- En todo caso queremos minimizar la función objetivo:

$$W^* = \underset{W}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$n = \text{\#ejemplos}$

$$W^* = \underset{W}{\operatorname{argmin}} J(W)$$

Función de costo

Recordar: son los pesos empleados en todas las capas de la red:
 $W = \{W^{(0)}, W^{(1)}, \dots\}$

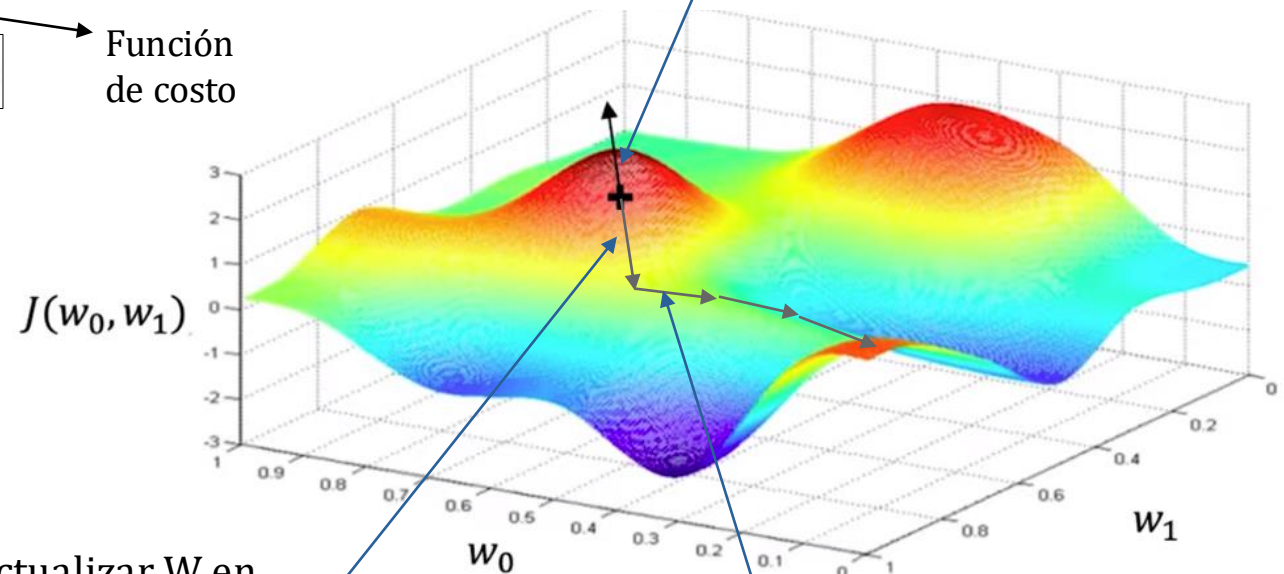


2. Actualizar W en dirección negativa del gradiente:

$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

η
Learning rate

1. Computar gradiente: $\frac{\partial J(W)}{\partial W}$



3. Iterar hasta convergencia

Ilustración y efecto learning rate ([enlace](#))

- ▶ Corrección del algoritmo GD para trabajar con **batches** (subconjuntos pequeños de ejemplos de entrenamiento):
 - Más eficiente que trabajar con todo el *dataset* a la vez
 - Menos ruidoso que trabajar con elementos de *training* individuales
- ▶ **Algoritmo SGD** (*stochastic gradient descent*):
 1. Inicializar pesos aleatoriamente $\sim \mathcal{N}(0, \sigma^2)$
 2. Iterar hasta convergencia (*batch a batch*; y haciendo varias pasadas por todo el conjunto de *training*):
 3. Computar gradiente de función objetivo: $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$

B = tamaño *batch*

↓
 4. Actualizar pesos: $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
 5. Devolver pesos óptimos

Redes neuronales para visión artificial

Algoritmo gradiente descendente



► Algoritmo:

- 1) Inicializar pesos aleatoriamente: $\sim \mathcal{N}(0, \sigma^2)$
- 2) Iterar hasta convergencia:
- 3) Computar gradiente de función objetivo: $\frac{\partial J(W)}{\partial W}$
- 4) Actualizar pesos: $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- 5) Devolver pesos óptimos.



```
import tensorflow as tf

weights = tf.Variable([tf.random.normal()])

while True:    # loop forever
    with tf.GradientTape() as g:
        loss = compute_loss(weights)
        gradient = g.gradient(loss, weights)

    weights = weights - lr * gradient
```



¡Hasta varios días de cómputo en problemas grandes!

- 1) Dependiendo de la complejidad de la red...
- 2) ...la cantidad y complejidad de los datos de entrenamiento...
- 3) ... y por supuesto el *hardware*.

Redes neuronales para visión artificial

Procedimiento completo con *Tensorflow*



```
import tensorflow as tf

model = tf.keras.Sequential([...])

# pick your favorite optimizer
optimizer = tf.keras.optimizer.SGD()

while True: # loop forever

    # forward pass through the network
    prediction = model(x)

    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)

    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

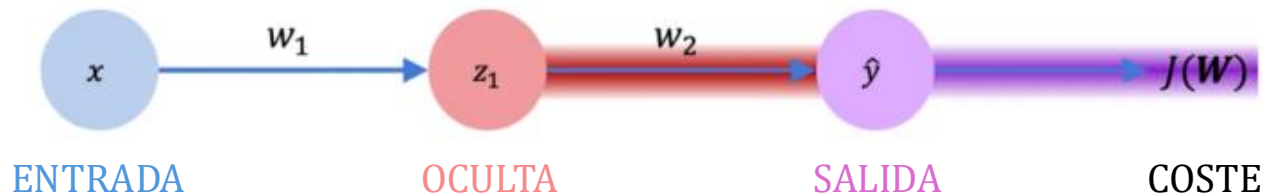
1. Construir modelo
(grafo computacional)
2. Decidir optimizador
(variante concreta *backprop*)
3. Pasar cada *batch* de ejemplos
por el modelo (*fwd pass*)
4. Estimar el gradiente
del costo respecto a los
parámetros (pesos modelo)
5. Actualizar pesos

Redes neuronales para visión artificial

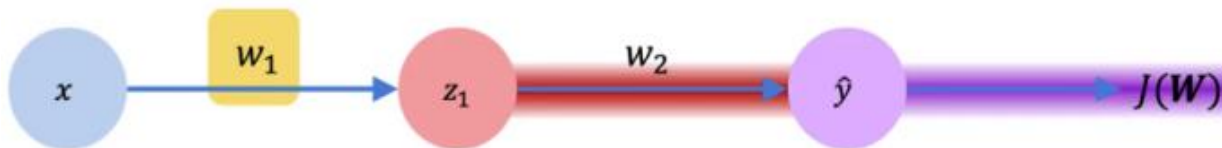
Cómputo gradiente: *backpropagation*



► ¿Cómo afecta cada pequeño cambio en w_i al costo $J(W)$?

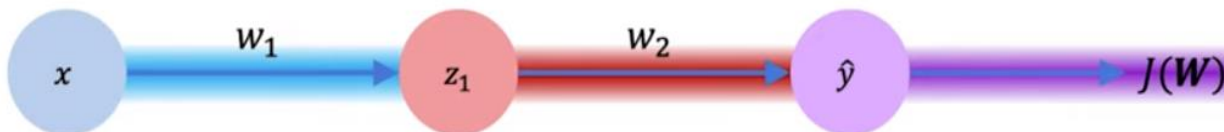


$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

Regla de la cadena



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Animated Math Channel [video](#)

Redes neuronales para visión artificial

Grafos computacionales (I)



Máster en
Inteligencia
Artificial
UMU

Ejemplo simple de *backpropagation*:

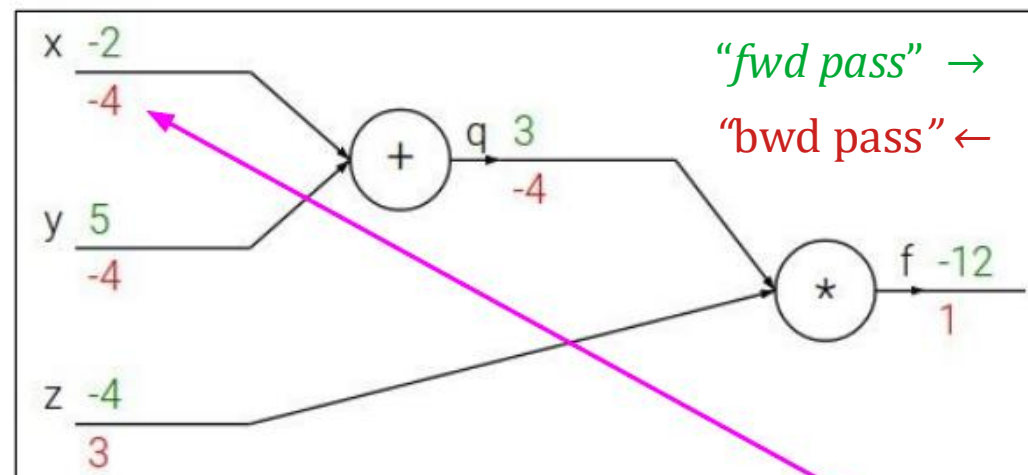
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Queremos: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$



"fwd pass" →

"bwd pass" ←

$$\frac{\partial f}{\partial x}$$

Regla de la cadena:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

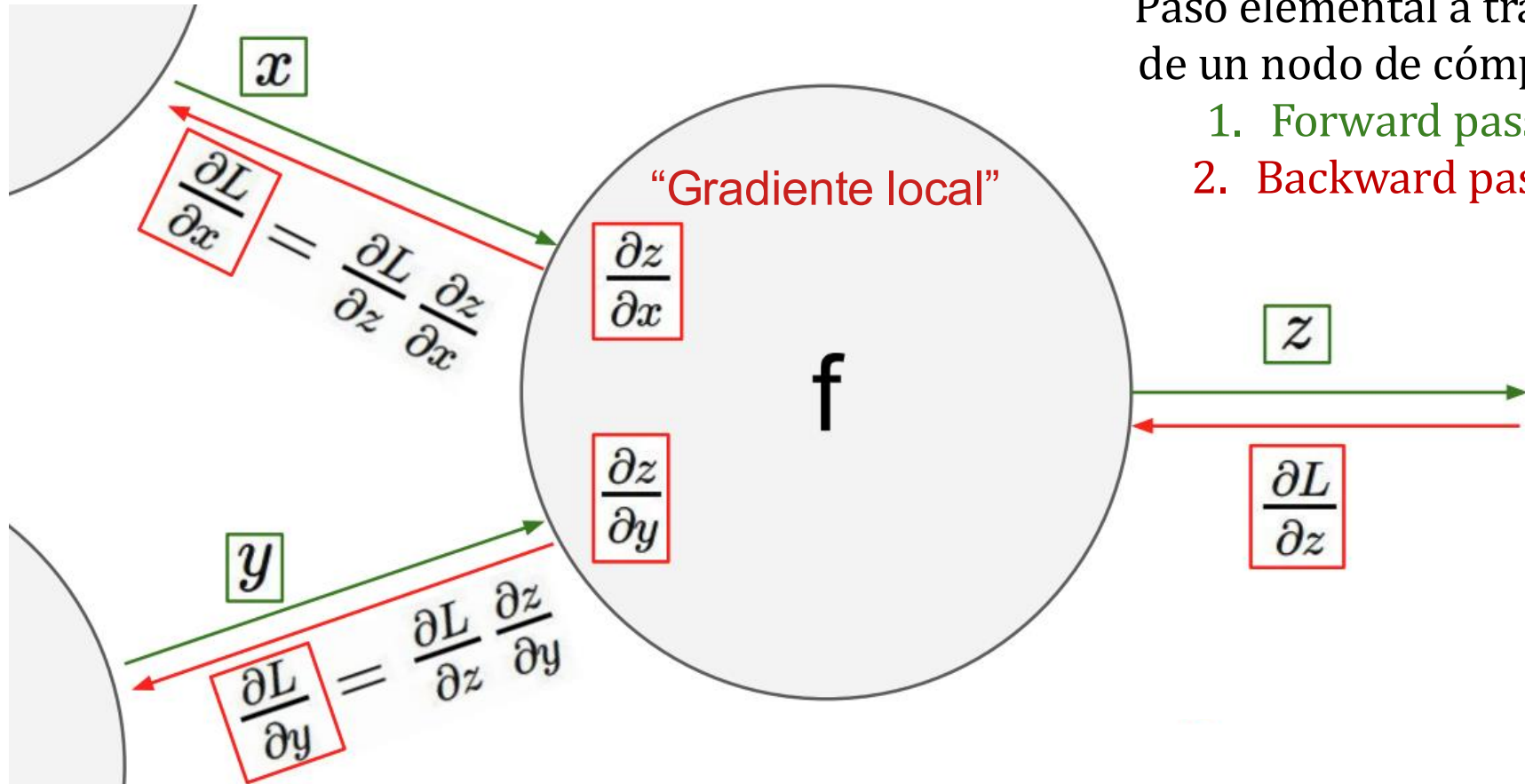
Redes neuronales para visión artificial

Grafos computacionales (II)



Situación general:
Paso elemental a través
de un nodo de cómputo

1. Forward pass
2. Backward pass



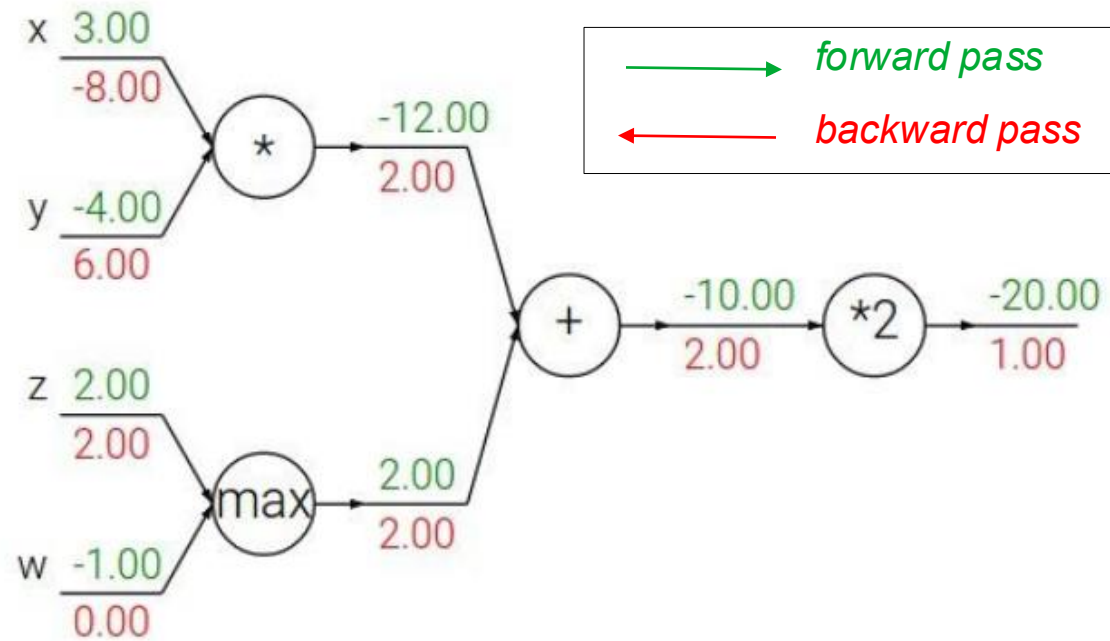
Redes neuronales para visión artificial

Grafos computacionales (III)



► Patrones en el flujo hacia atrás (*backward pass*), según operación:

- Nodo multiplicación por constante: multiplica gradiente por constante.
- Nodo suma de nodos: distribuye gradiente.
- Nodo multiplicación de nodos: multiplica gradiente por entradas intercambiadas.
- Nodo máximo de nodos: “enruta” el gradiente.
- ...



Redes neuronales para visión artificial

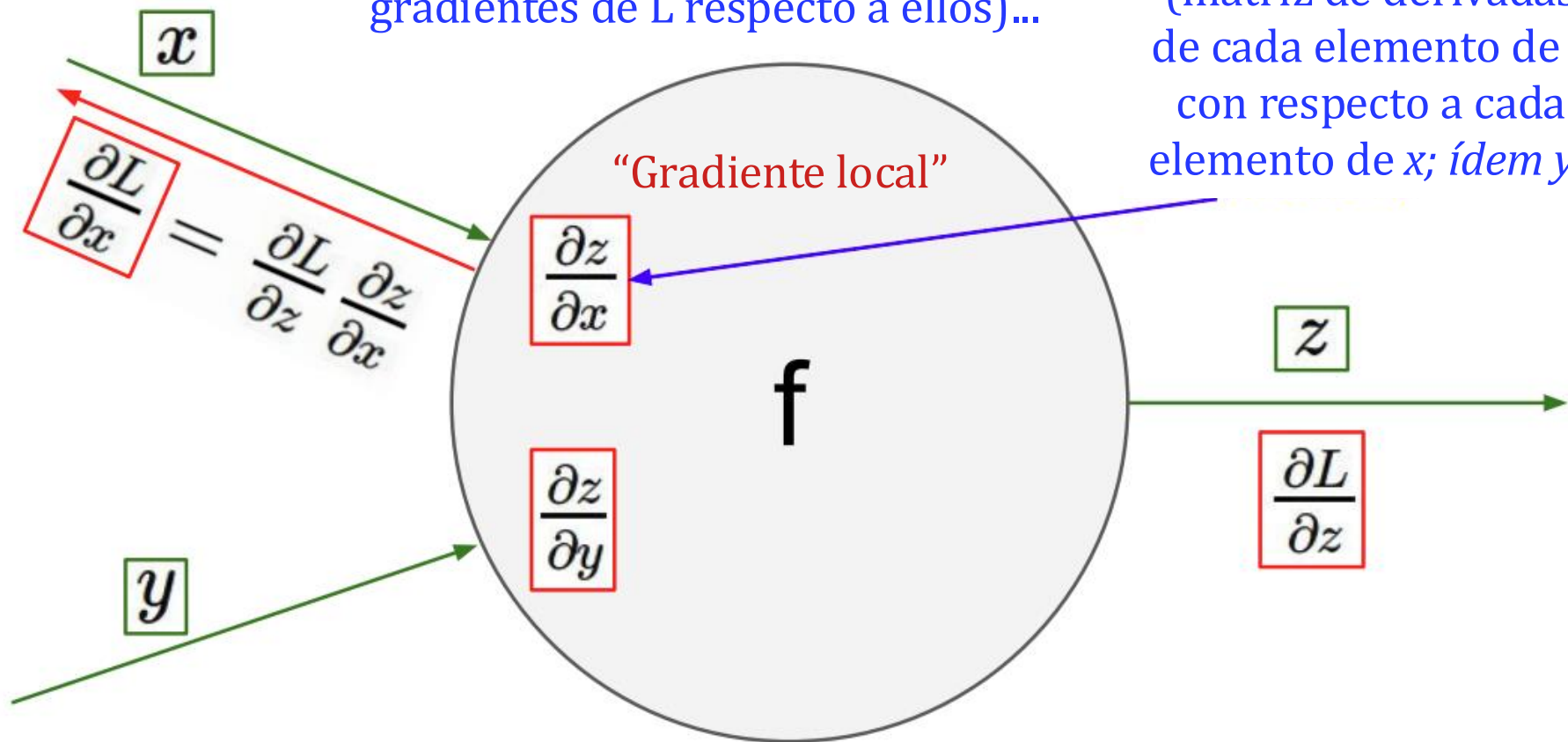
Grafos computacionales (IV)



► Grafos para código vectorizado:

x, y, z son ahora vectores (igual que los gradientes de L respecto a ellos)...

... mientras que esto ahora son jacobianos (matriz de derivadas de cada elemento de z con respecto a cada elemento de x ; ídem y)



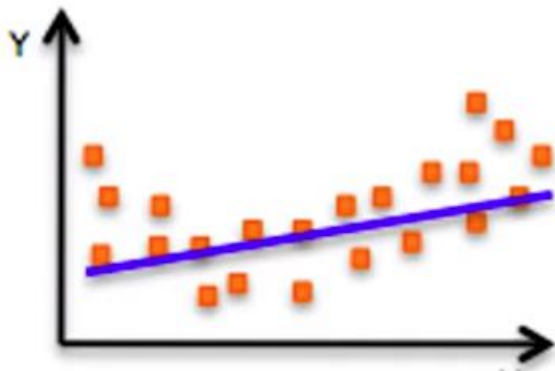
https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html

Redes neuronales para visión artificial

Posible problema: sobreajuste

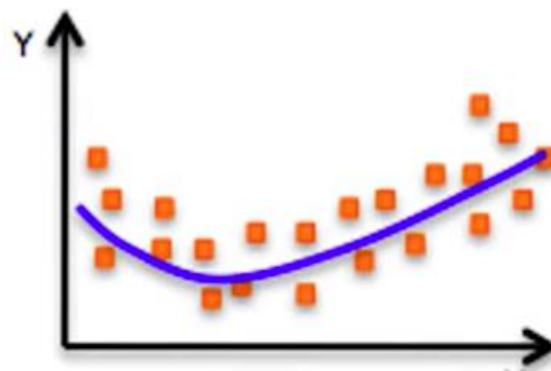


- Al ajustar una red (modelo) a unos datos (*training dataset*) podemos experimentar el problema conocido como **overfitting** (especialmente si se tienen muchos parámetros ajustables, como es típico en DL):

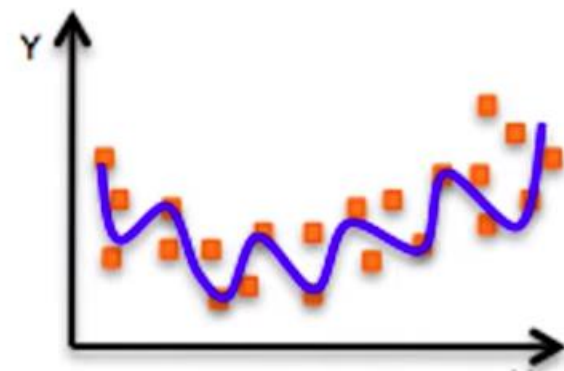


Underfitting

El modelo no tiene capacidad suficiente para ajustarse bien a los datos



Ajuste ideal



Overfitting

El modelo tiene demasiada capacidad, sobreajusta y no generaliza bien

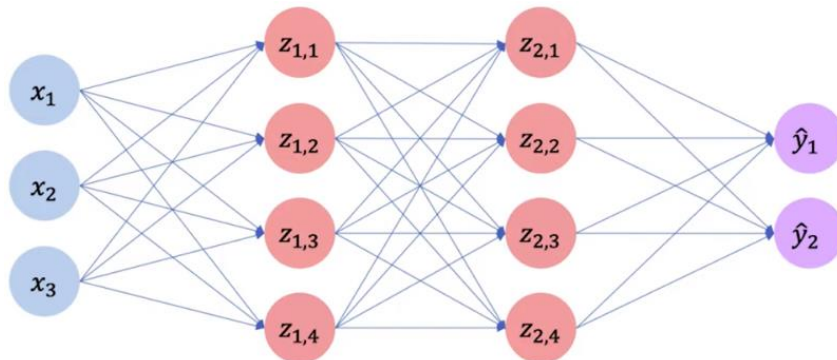
↓
Necesitamos algún tipo de
regularización

Redes neuronales para visión artificial

Regularización I: *dropout*

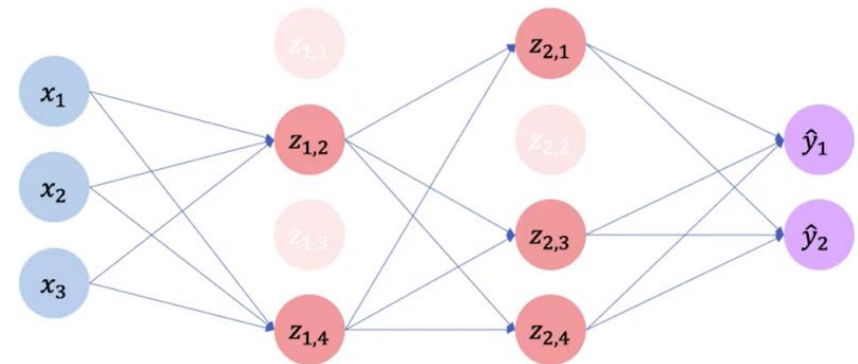
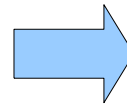


- ▶ Modelo debe funcionar con menos parámetros → disminuye capacidad → atenúa **overfitting**.
- ▶ Eliminar (aleatoriamente) un determinado porcentaje de activaciones (hacerlas cero) durante el aprendizaje:

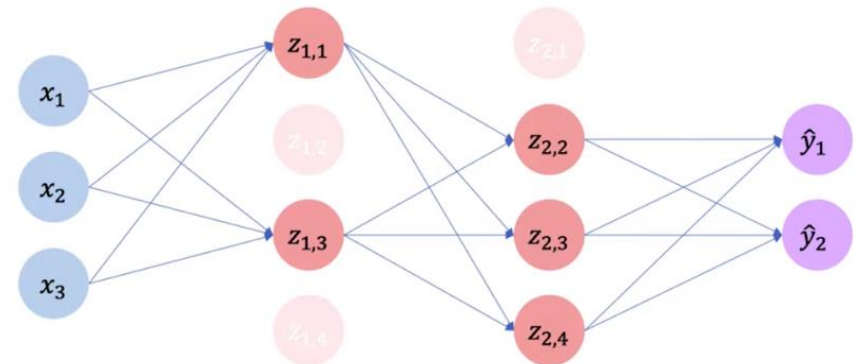


Se fuerza a que **la red no dependa demasiado de cada nodo** individualmente

```
 tf.keras.layers.Dropout(p=0.5)
```



... en cada iteración se eliminan activaciones diferentes ...



... ¡Pero ojo, en inferencia se usan todos los pesos aprendidos!

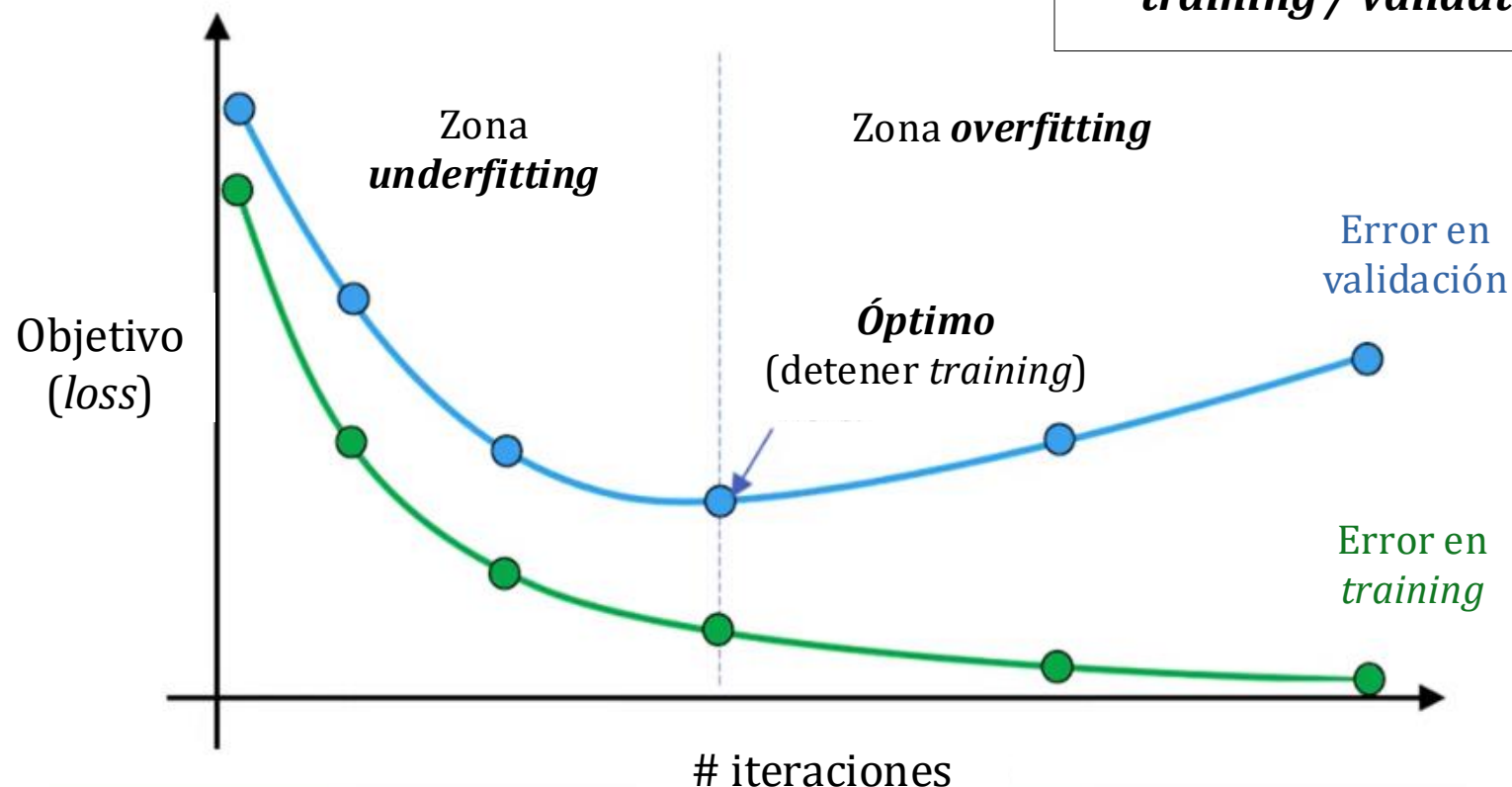
Redes neuronales para visión artificial

Regularización II: *early stopping*



- Detener aprendizaje cuando no mejore la pérdida en el conjunto de ejemplos de **validación** (aunque siga mejorando en el de entrenamiento):

Muy importante:
separación previa de *dataset* en
training / *validation* / *test*



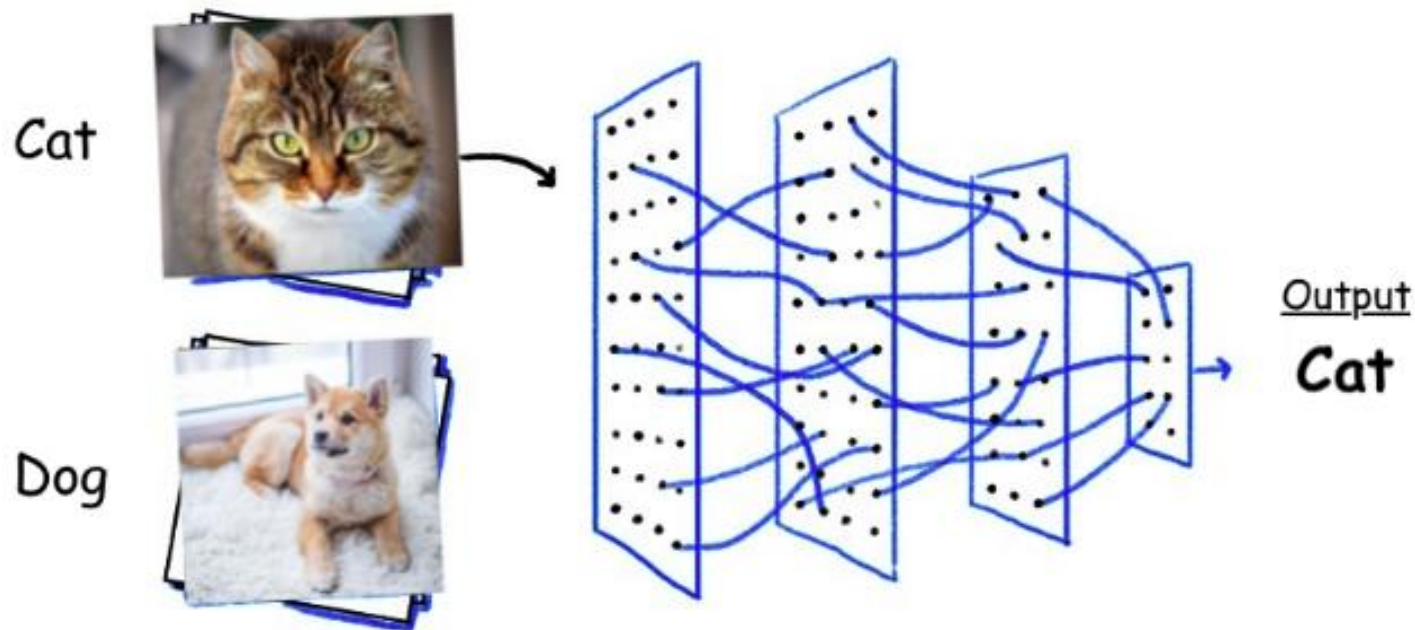
Redes neuronales para visión artificial

Redes neuronales convolucionales (CNNs)



► Problema: clasificar imágenes:

- A partir de conjuntos de imágenes etiquetados con una categoría, diseñar y entrenar una red capaz de obtener la etiqueta correcta para imágenes nuevas (no empleadas en el entrenamiento):



Redes neuronales para visión artificial

CNNs para clasificación: *ImageNet*



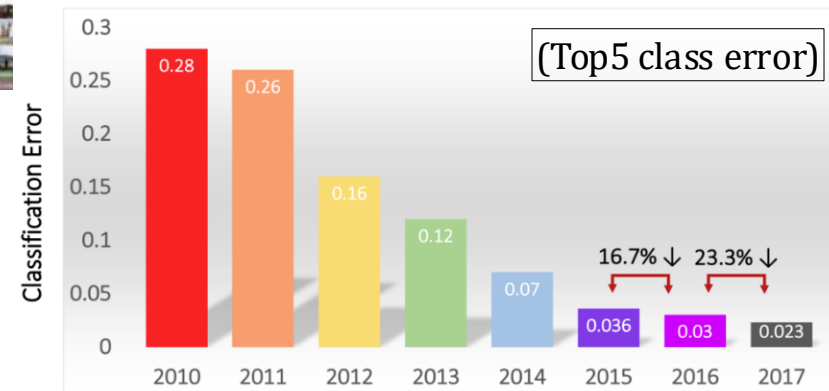
Máster en
Inteligencia
Artificial
UMU

► **ImageNet-21k** (~14M imgs etiquetadas, ~21000 clases):



<http://www.image-net.org/>

Classification Results (CLS)



Alexnet → 1ª GPU CNN

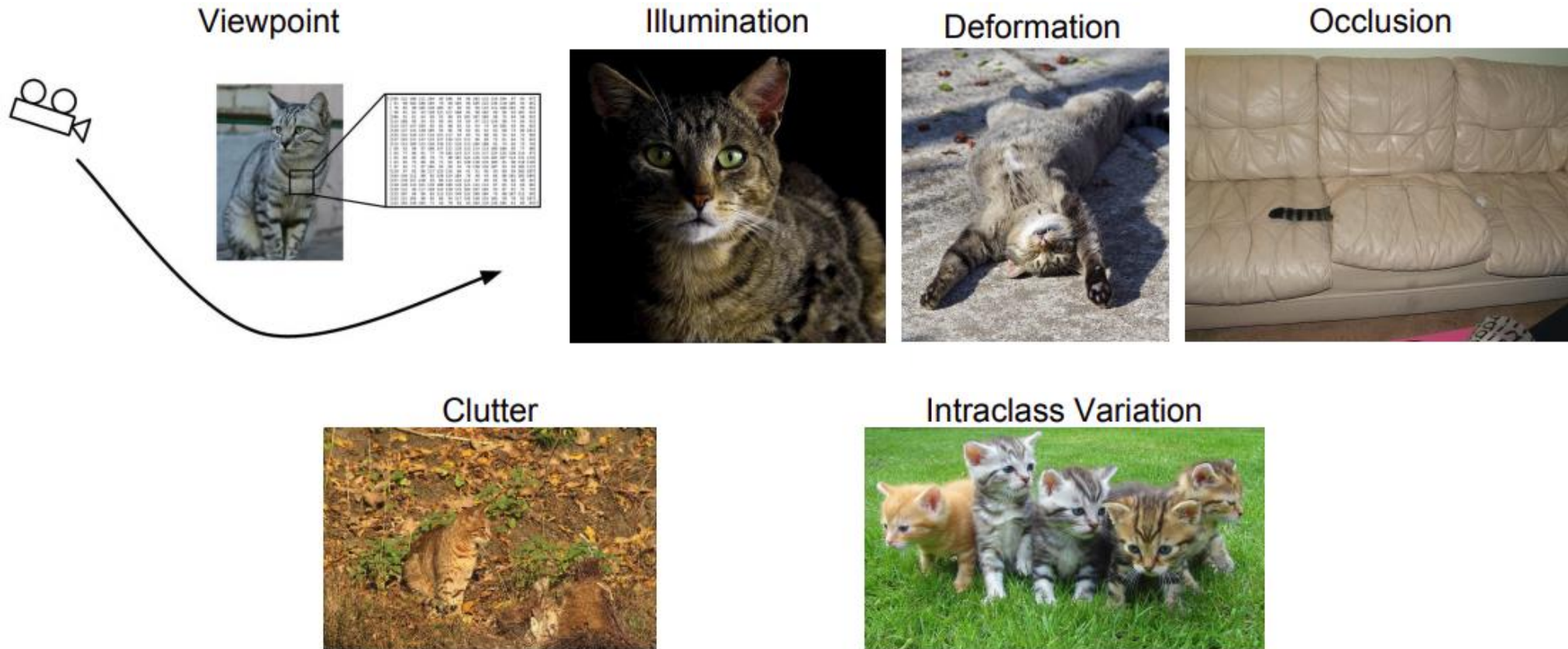
Redes neuronales para visión artificial

CNNs para clasificación: Desafíos



Máster en
Inteligencia
Artificial
UMU

► Variabilidad extrema:



¡(Casi) nada de lo visto hasta ahora en el curso
sirve en estas condiciones tan difíciles!

Redes neuronales para visión artificial

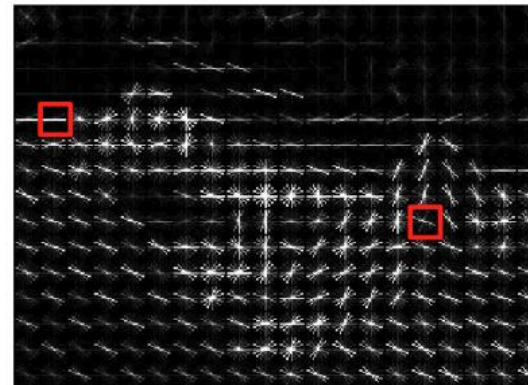
Intentos de soluciones previas



- ▶ Ejemplo de vectorización alternativa a la “imagen cruda”: Histogramas de gradientes. **HoGs**:



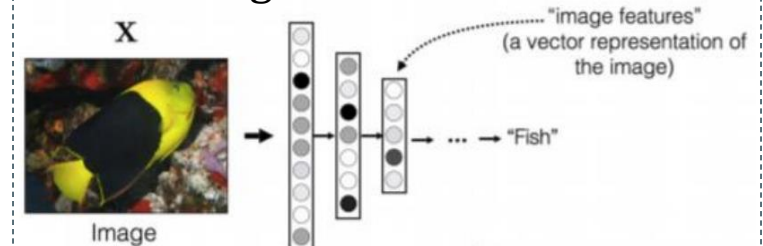
Divide image into 8x8 pixel regions
Within each region quantize edge
direction into 9 bins



Example: 320x240 image gets divided
into 40x30 bins; in each bin there are
9 numbers so feature vector has
 $30 \times 40 \times 9 = 10,800$ numbers

- ▶ En el fondo, idea similar a los descriptores SIFT, pero para toda la imagen, pero...
- ▶ ... problemas de alineamiento previo, escala, etc.
- ▶ Además, ¿son los HoG suficientemente similares intraclase?

Recordar Tema 1: Representar
una imagen como un vector

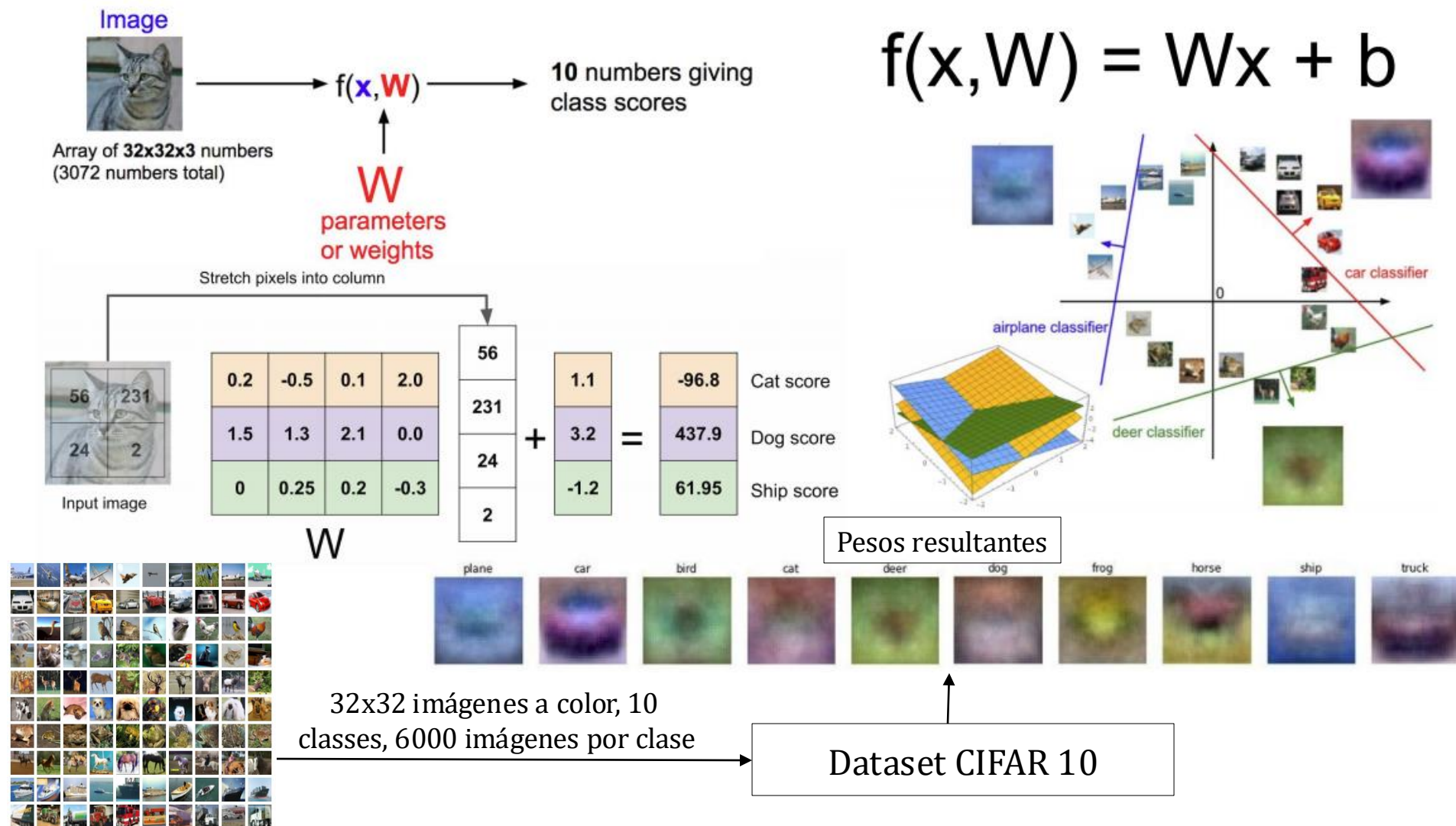


Redes neuronales para visión artificial

Un primer modelo neuronal para imágenes (I)



- Esquema intuitivo de cómo sería un **simple clasificador lineal** (sin capa oculta):



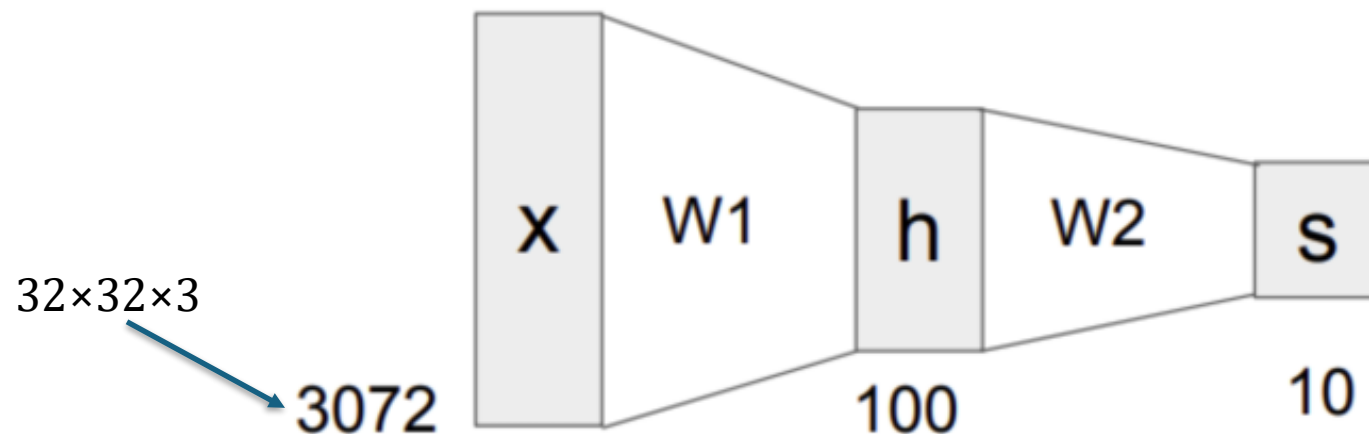
Redes neuronales para visión artificial

Un primer modelo neuronal para imágenes (II)



► Red neuronal no convolucional más sencilla:

- **1ª capa:** Como un clasificador lineal, pero... con una no linealidad en la salida: $\text{ReLu}(\cdot) = \max(\cdot, 0)$.
- **2ª capa:** Como un clasificador lineal, pero... sobre las salidas (ya no linealizadas) de la primera capa.



- Clasificación lineal simple: $f = W \cdot x$
- NN con no linealidad oculta: $f = W_2 \max(0, (W_1 \cdot x))$

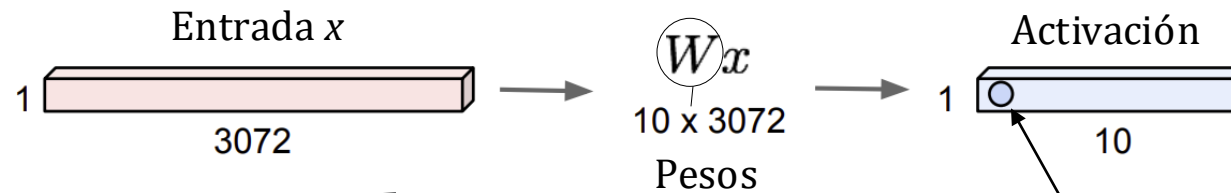
Redes neuronales para visión artificial

Capa FC vs. capa convolucional (I)



1. Capa FC (*fully connected*, completamente conectada):

Imagen $32 \times 32 \times 3 \rightarrow$ vector 3072×1

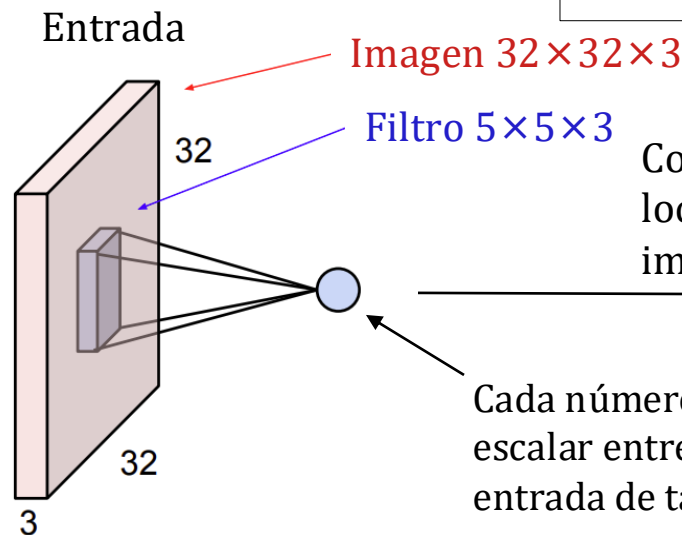


Cada número es el resultado de un producto escalar entre una fila de W y el vector de entrada 3072×1 (+ un *bias*)

En ambos casos, $w^T x + b$

2. Capa convolucional:

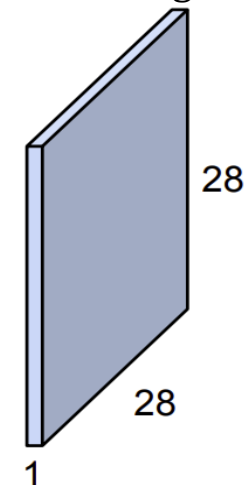
Se repite el mismo cómputo sobre toda la imagen



Convolucionar sobre todas las localizaciones espaciales de la imagen (desplazándose por ella)

Cada número es el resultado de un producto escalar entre el filtro y un subbloque de entrada de tamaño $5 \times 5 \times 3$ (+ un *bias*)

“Mapa de activación”
(entrada a la siguiente capa)



Redes neuronales para visión artificial

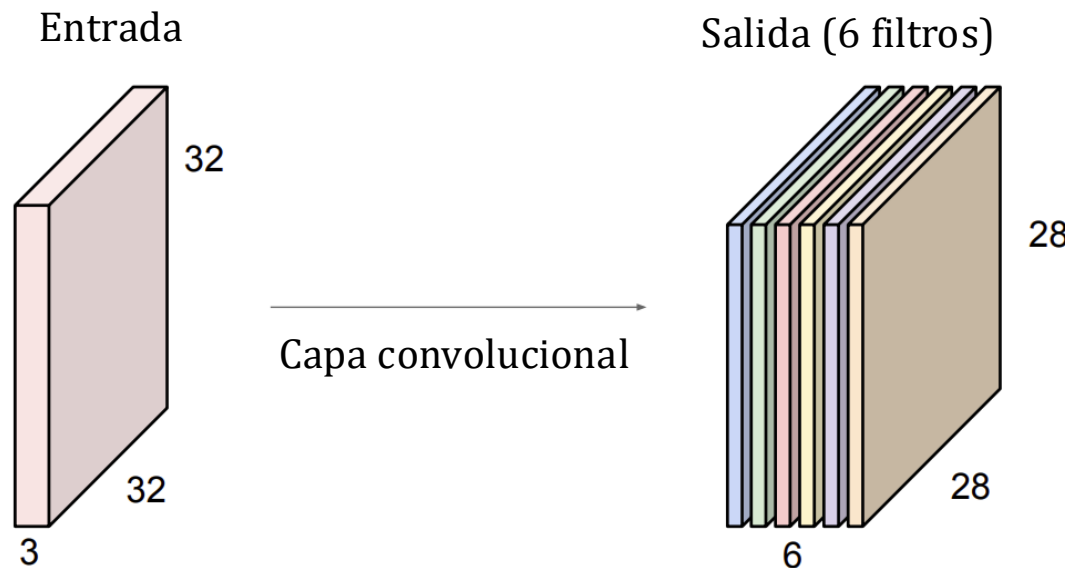
Capa FC vs. capa convolucional (II)



Vamos añadiendo más filtros a la capa...



... con lo que simplemente se aumenta la “profundidad” de la salida de la capa (mapa de activación)



Obsérvese que se reduce el número de parámetros: de $(32 \times 32 \times 3) \times (28 \times 28 \times 6) + 1 \approx 14\text{M}$ de una capa FC a sólo $(5 \times 5 \times 3 + 1) \times 6 = 456$ del enfoque convolucional →

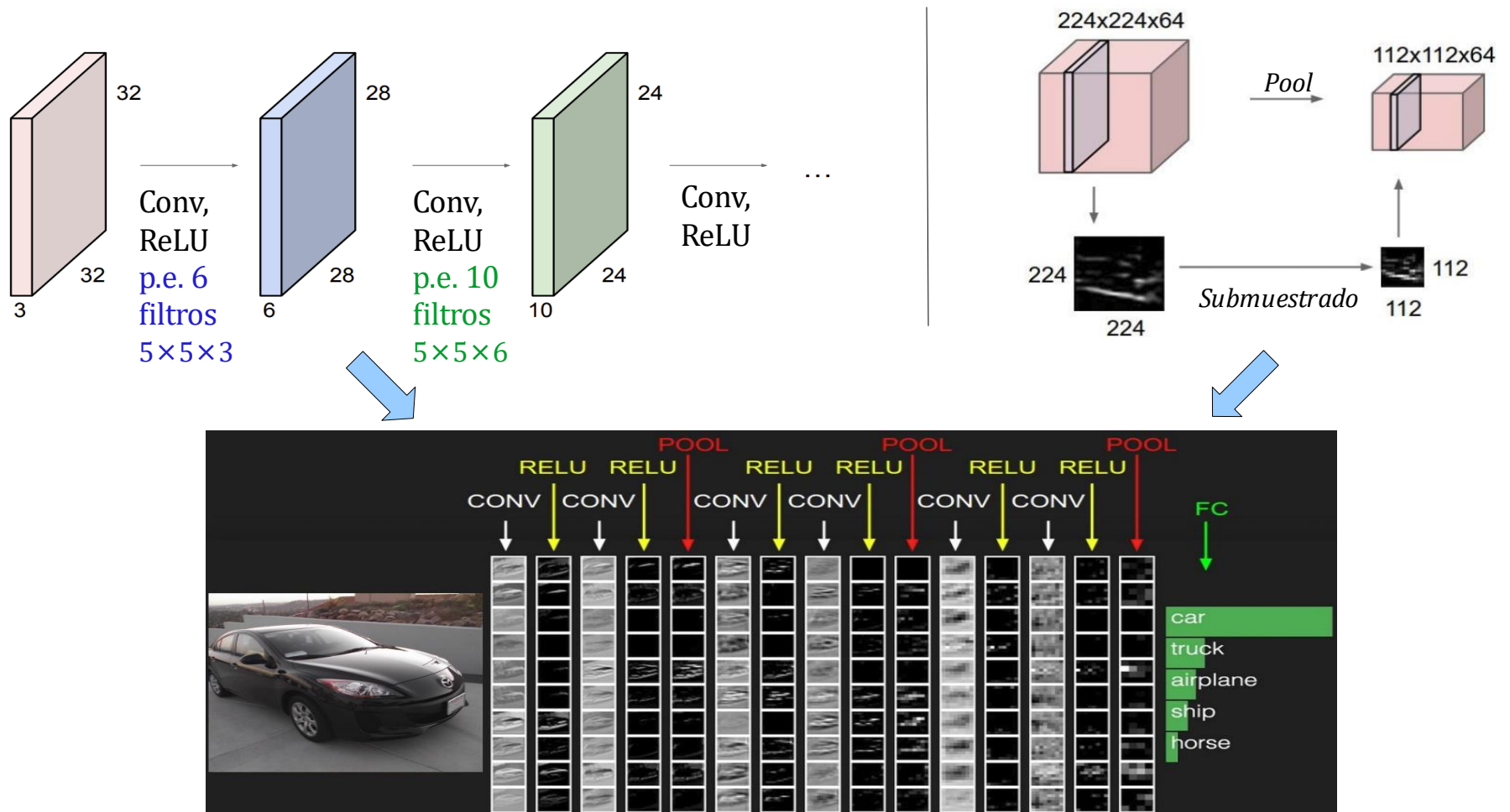
“receptive field” más limitado

Redes neuronales para visión artificial

Capa FC vs. capa convolucional (III)

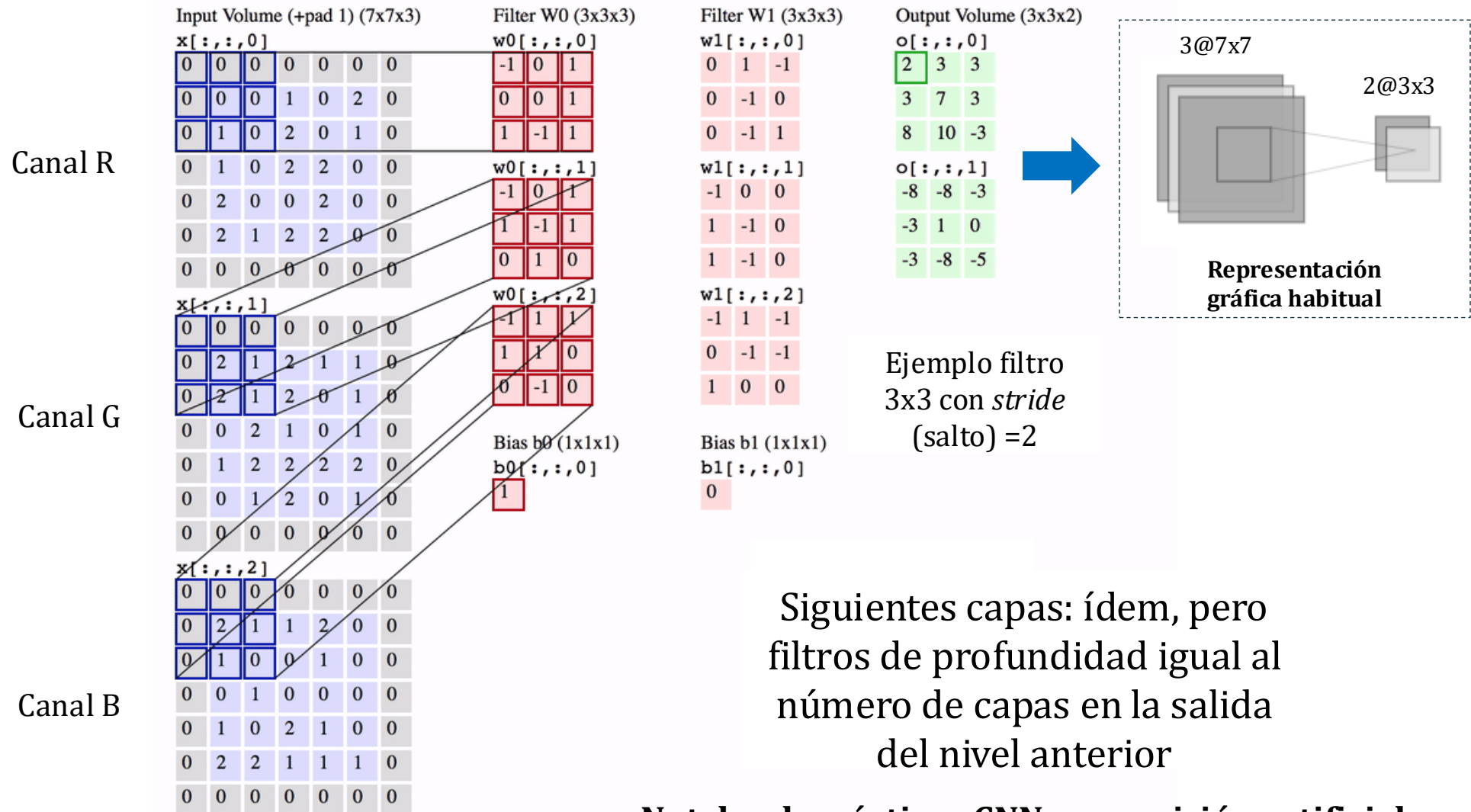


- Una CNN es simplemente una secuencia de capas convolucionales, entre las cuales simplemente se introducen funciones de activación (ReLU) / reducción (pooling); éste último de tipo *average* o *max* (más común):



Redes neuronales para visión artificial

Acción de los pesos en cada capa



Siguientes capas: ídem, pero
filtros de profundidad igual al
número de capas en la salida
del nivel anterior

Notebook prácticas CNNs para visión artificial

VC_using_CNNs.ipynb

[Enlace animación](#)

Redes neuronales para visión artificial

Padding & stride en convoluciones (I)



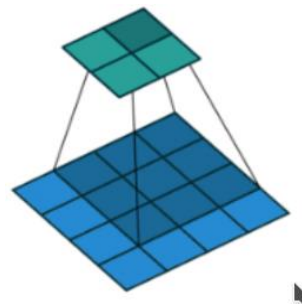
Máster en
Inteligencia
Artificial
UMU

► **Convoluciones directas** (luego veremos las traspuestas):

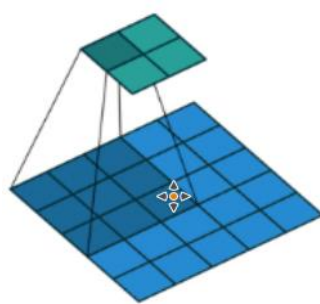
- **Padding** → Tratamiento en bordes
- **Strides** → “Salto en píxeles de entrada”

► Ejemplos:

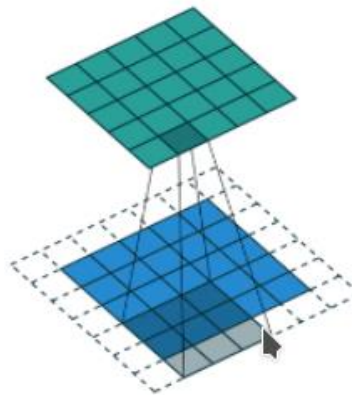
- **Azul** → Imagen de entrada
- **Verde** → Imagen de salida
- **Gris** → Máscara de convolución



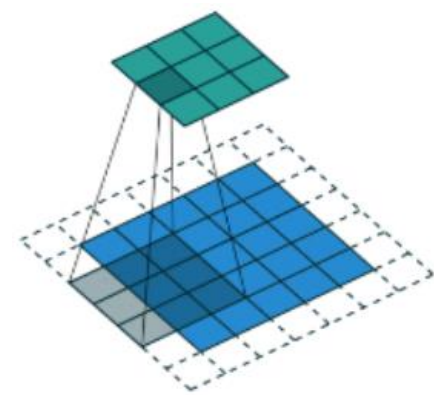
No padding,
stride=1
“VALID”



No padding,
stride=2



Padding,
stride=1
“SAME”



Padding,
stride=2

https://github.com/vdumoulin/conv_arithmetic

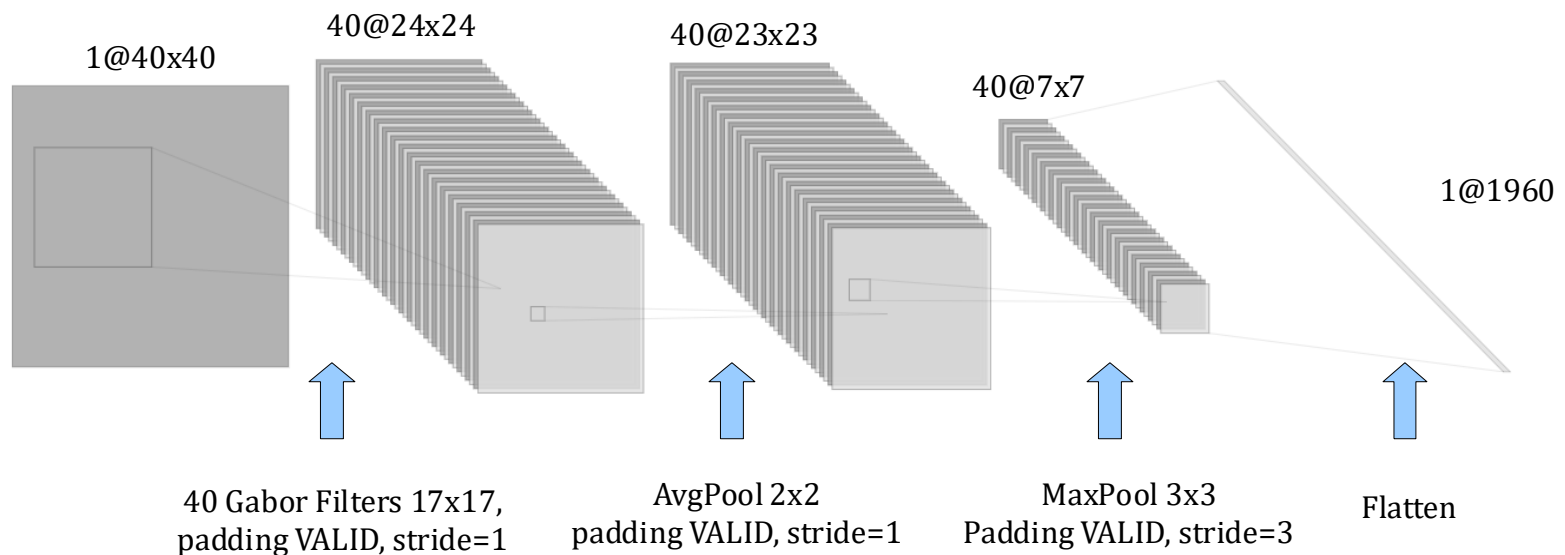
Redes neuronales para visión artificial

Padding & stride en convoluciones (II)



Máster en
Inteligencia
Artificial
UMU

► Ejemplo red notebook prácticas: (ALOTextures.ipynb, variante nivel de gris):



Redes neuronales para visión artificial

Algunas convoluciones especiales



► Convoluciones transpuestas:

- “Revierten” la convolución normal
- Utilizadas para “supermuestrear” (aumentar resolución del tensor de salida respecto al de entrada)

Convolución transpuesta →

A partir de una *feature* de entrada se computan varias de salida

55	52
57	50

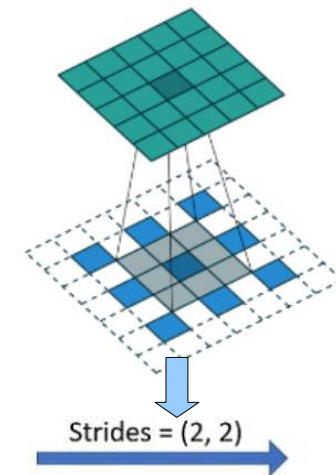
1	2
2	1



55	110	52	104
110	55	104	52
57	114	50	100
114	57	100	50

55	110	52	104
110	55	104	52
57	114	50	100
114	57	100	50

4x4 Output Matrix



Convolución normal →

A partir de varias *features* de entrada se computa una de salida

3	5	2	7
4	1	3	8
6	3	8	2
9	6	1	5

3	5	2	7
4	1	3	8
6	3	8	2
9	6	1	5

Strides = (1, 1)

1	2
2	1



3x3 Output Matrix

22	14	30
21	21	37
36	32	19

Ejemplo en Keras:

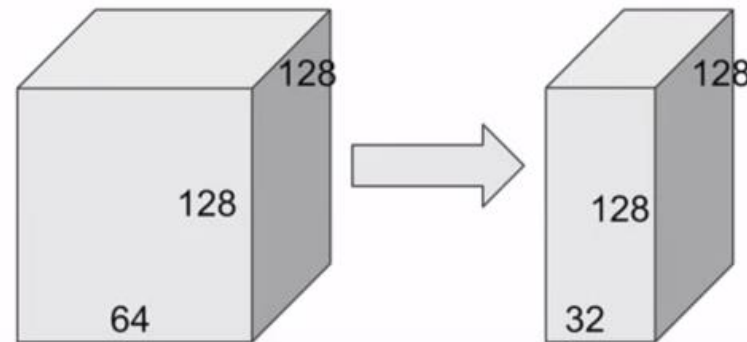
```
Conv2DTranspose(filters=32, kernel_size=(3, 3))
```

► Convoluciones 1x1:

- Utilizadas para “comprimir” el tensor de *features* en alguna(s) capa(s) intermedia(s) de la red:
- Cada *feature* (*slice* 1x1xF) en la entrada → *feature* 1x1xN en la salida (con $N < F$).
- Análoga a una FFN unidimensional F a N, repetida para todas las *features* del tensor de entrada.

(**B**, **F**, H, W) - B = # batches; F = # filters, H, W = Height/Width

(**B**, **N**, H, W) - B = # batches; N = # filters, H, W = Height/Width



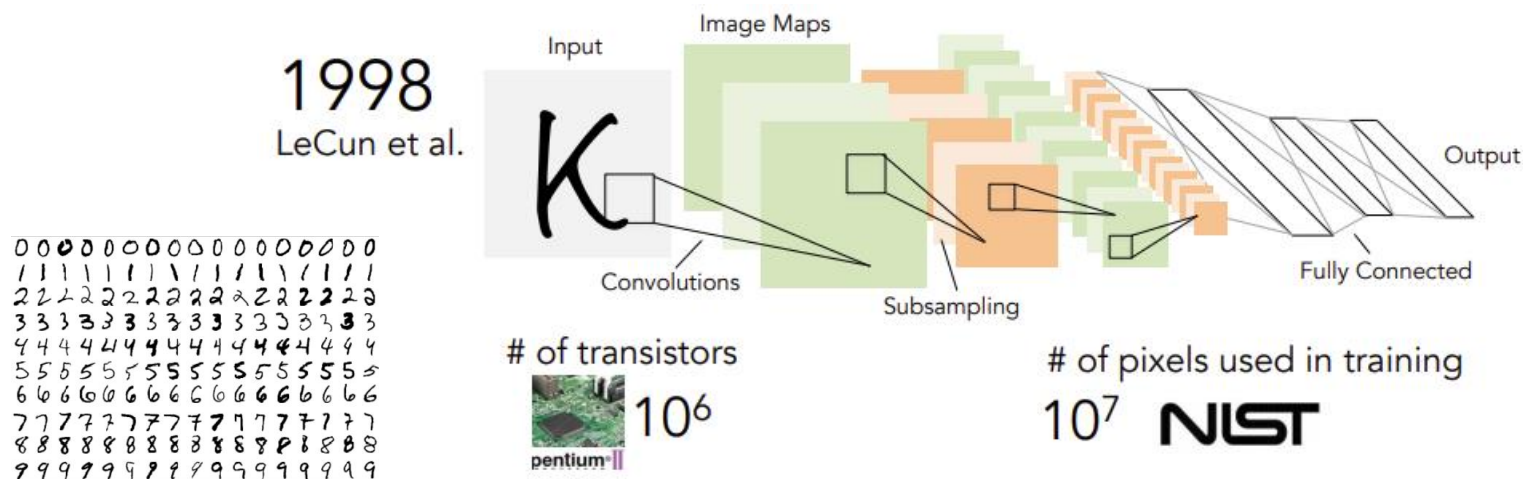
Redes neuronales para visión artificial

CNNs para clasificación: origen

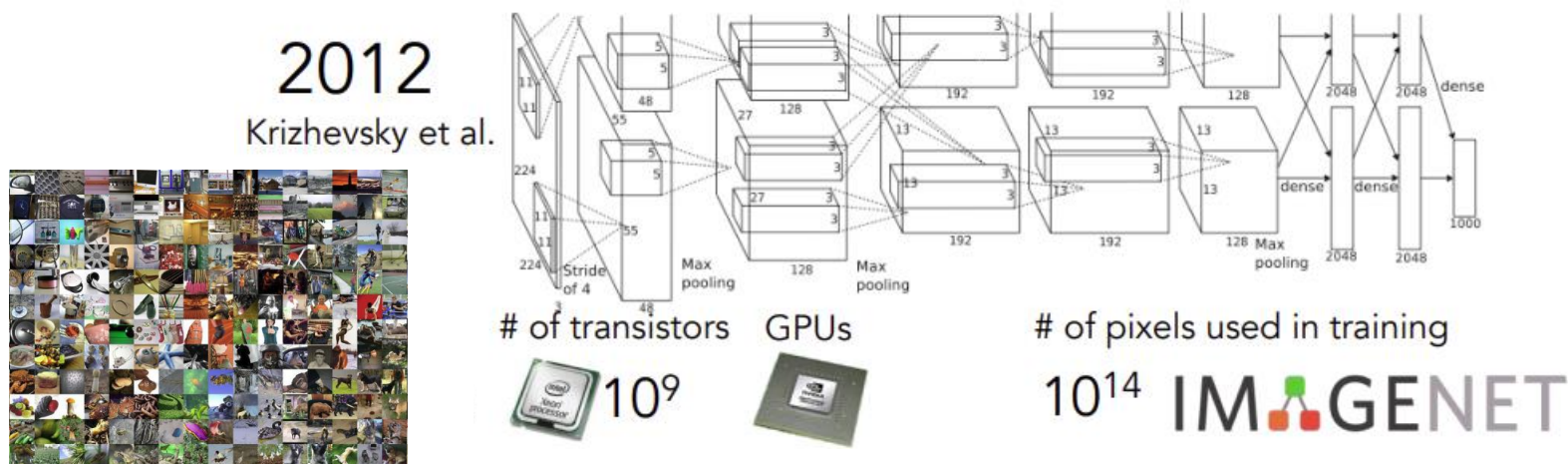


Máster en
Inteligencia
Artificial
UMU

► 1ª red CNN (LeNet; MNIST):



► 1ª red CNN profunda (AlexNet; ImageNet):

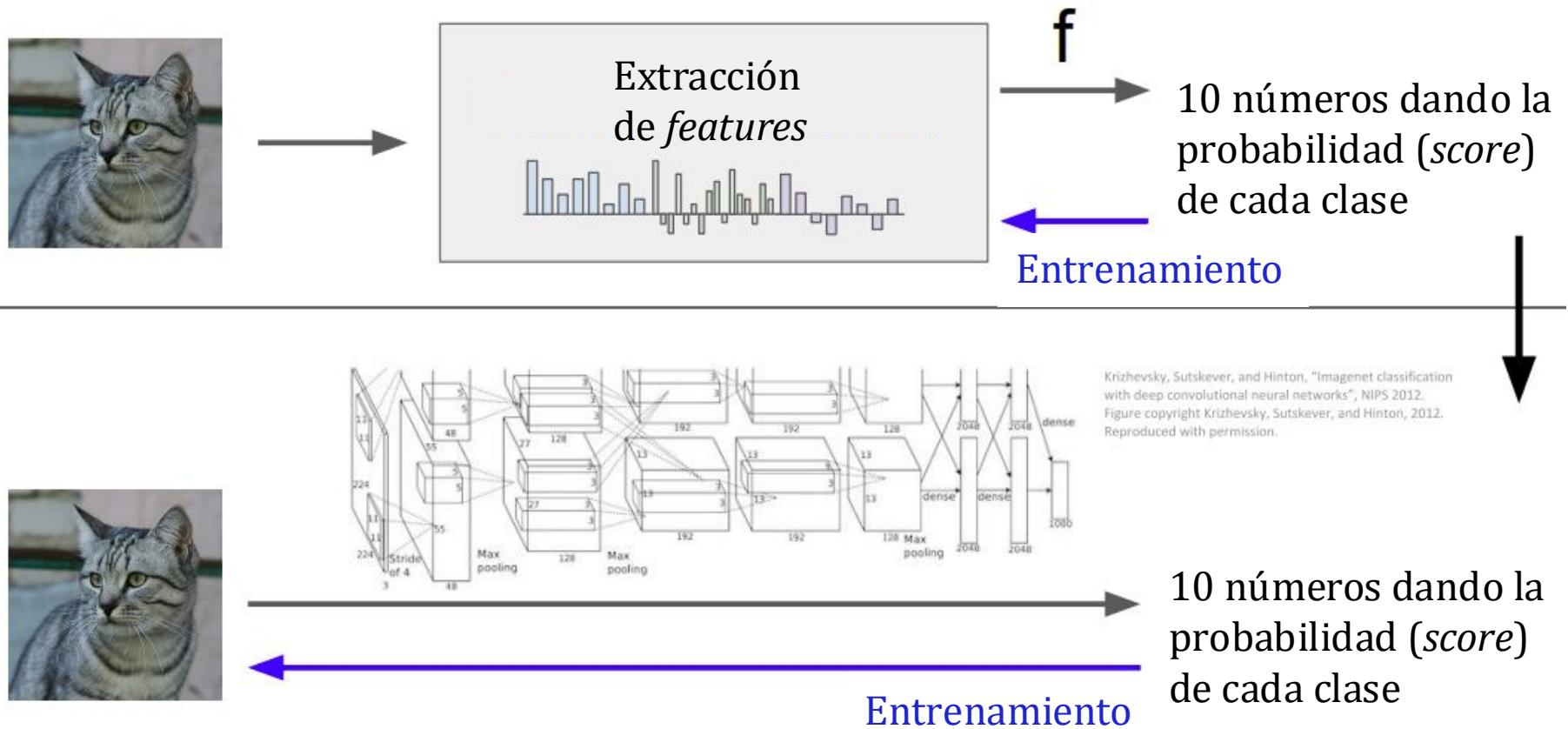


Redes neuronales para visión artificial

Engineered features vs. CNN features



- Dejamos que la propia red neuronal determine las características necesarias, a distintos niveles:



Notebook prácticas extracción manual vs automatizada:

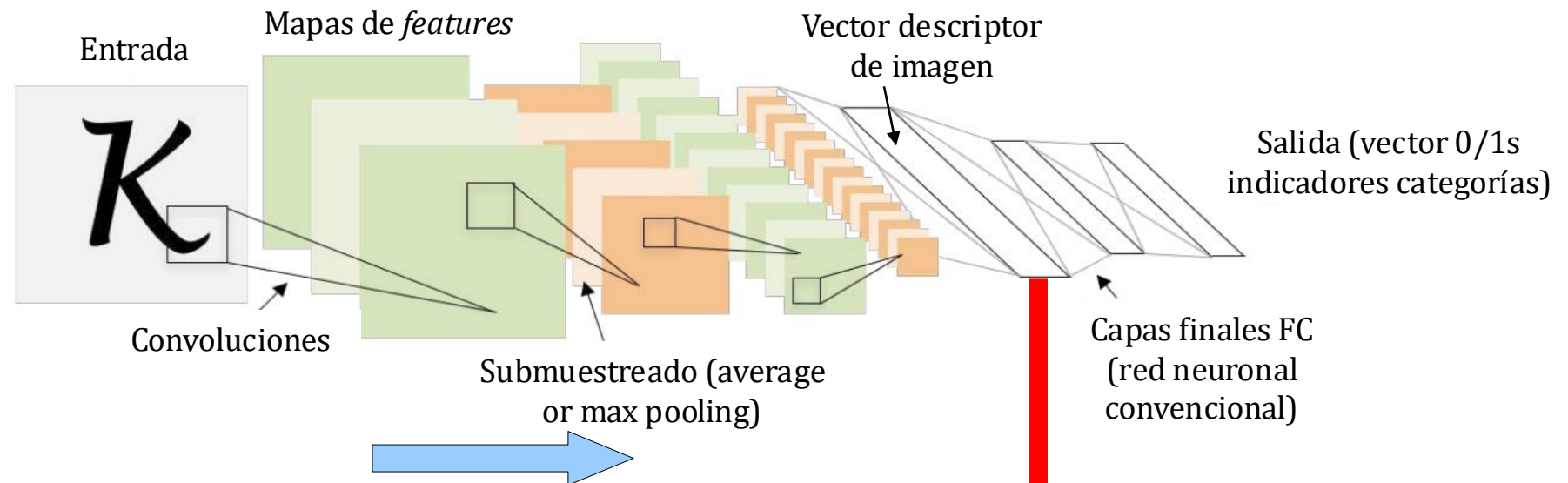
`ALOTexturesCNN.ipynb`

- ▶ Las CNNs serán grandes → No se hará el gradiente “a mano” → Automatización.
- ▶ *Backpropagation* → Regla de la cadena recursiva a través del grafo computacional.
- ▶ Dos pasadas:
 - **Forward**: Computar resultados de la operación de cada nodo en la CNN, y guardar resultados intermedios.
 - **Backward**: Aplicar la regla de la cadena para computar el gradiente de la función de pérdida (*loss function*) con respecto a las entradas.

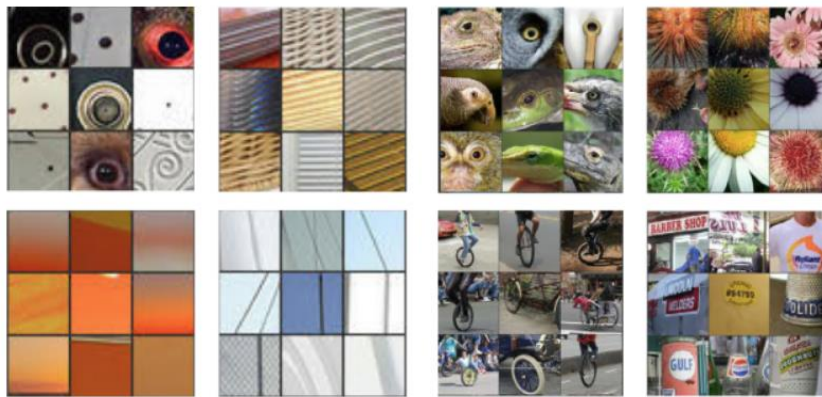
Misma filosofía que en una red NN tradicional

Redes neuronales para visión artificial

CNNs: “embudo perceptual”

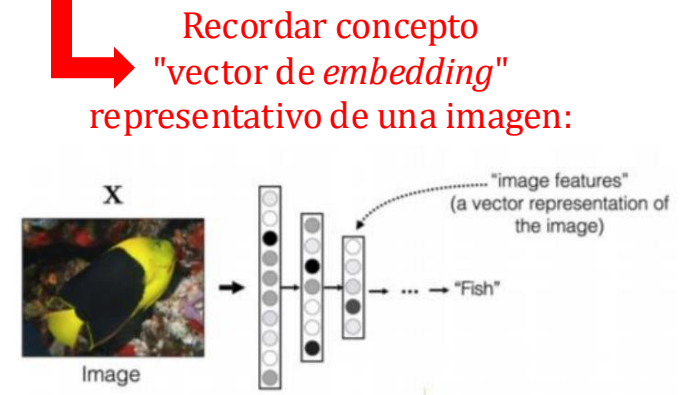


- Características progresivamente más complejas, abarcando *receptive fields* cada vez más grandes



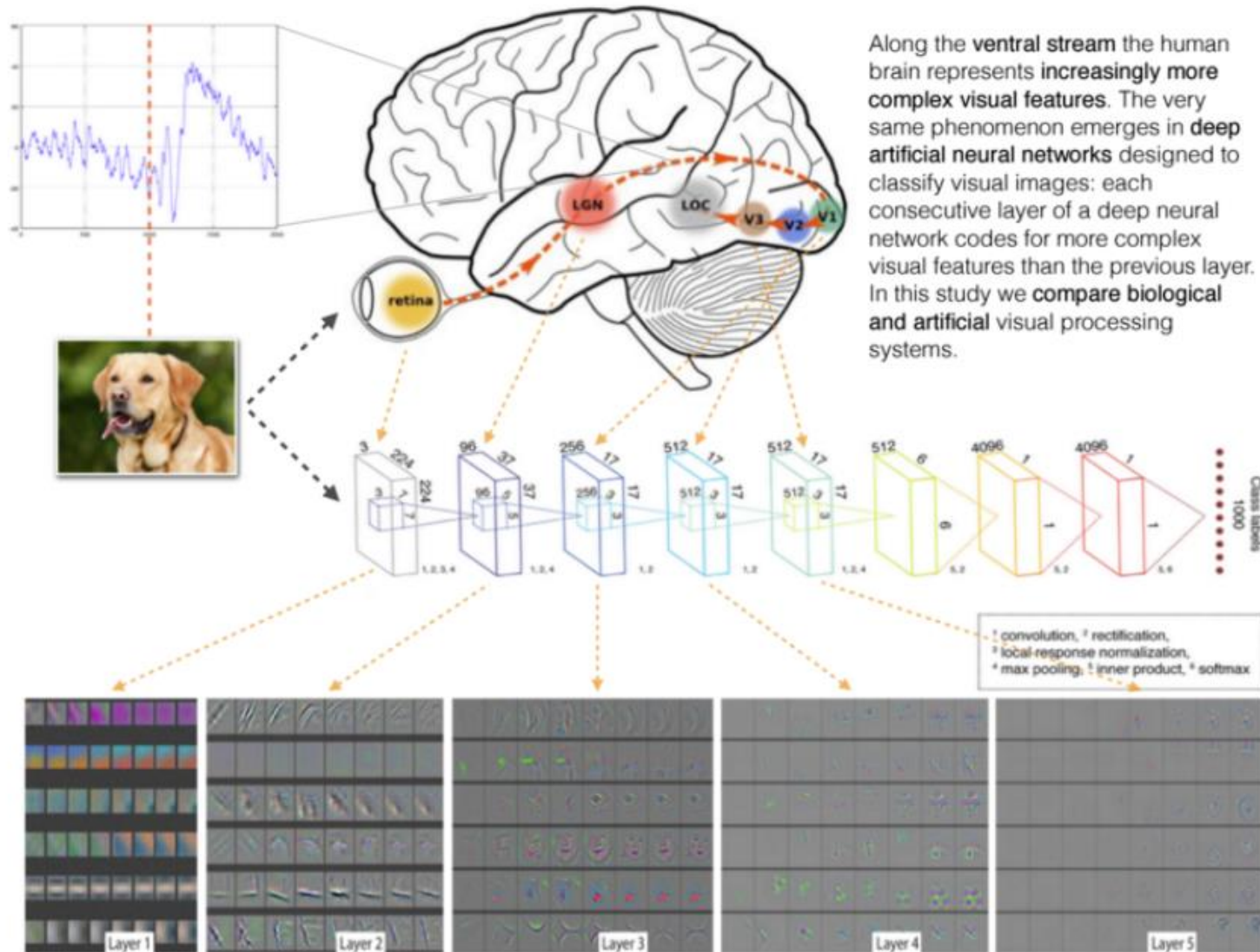
Features
bajo nivel

Features
alto nivel



Redes neuronales para visión artificial

Plausibilidad biológica (analogía niveles córtex cerebral)

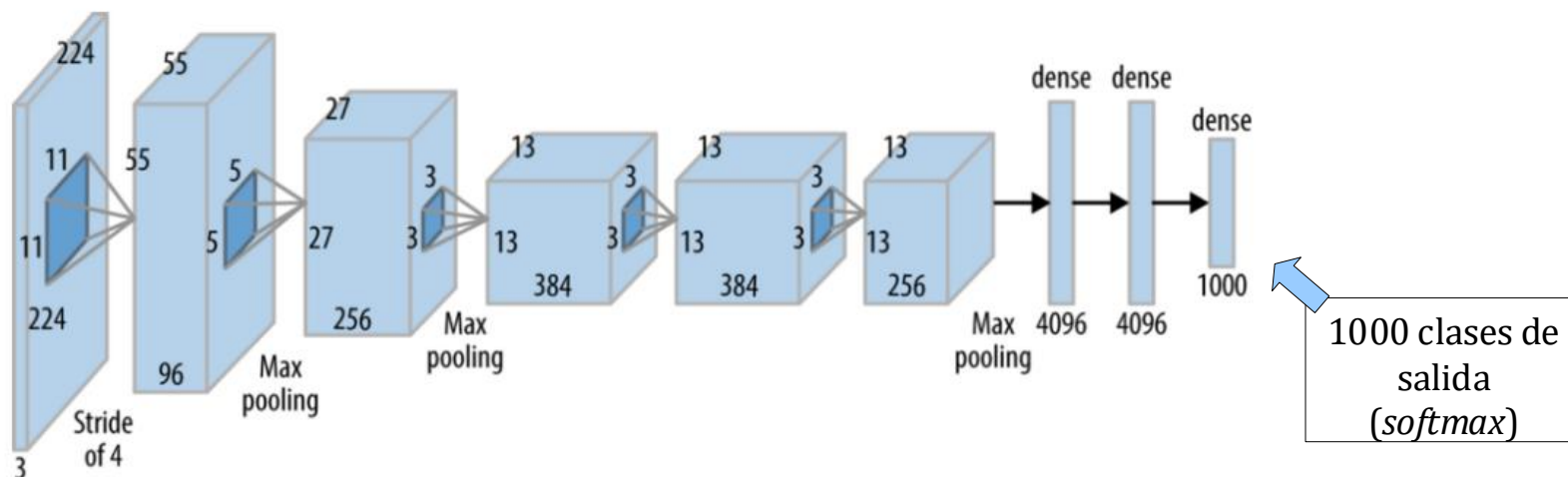


Redes neuronales para visión artificial

Número de parámetros (muy) elevado



- **AlexNet** (Krizhesky, Sutskever, Hinton, 2012, [link](#)):
 - Primera CNN “moderna”, considerada profunda (*deep*).



Input		Output		Layer	Stride	Pad	Kernel size		in	out	# of Param
224	224	3	55 55 96	conv1	4	0	11	11	3	96	34944
55	55	96	27 27 96	maxpool1	2	0	3	3	96	96	0
27	27	96	27 27 256	conv2	1	2	5	5	96	256	614656
27	27	256	13 13 256	maxpool2	2	0	3	3	256	256	0
13	13	256	13 13 384	conv3	1	1	3	3	256	384	885120
13	13	384	13 13 384	conv4	1	1	3	3	384	384	1327488
13	13	384	13 13 256	conv5	1	1	3	3	384	256	884992
13	13	256	6 6 256	maxpool5	2	0	3	3	256	256	0
6 x 6 x 256 = 9216				fc6			1	1	9216	4096	37752832
				fc7			1	1	4096	4096	16781312
				fc8			1	1	4096	1000	4097000
Total											62,378,344

$$11 \times 11 \times 3 \times 96 + 96 = 34944$$

$$5 \times 5 \times 96 \times 256 + 256 = 614656$$

...

$$9216 \times 4096 + 4096 = 37752832$$

$$(9216 = 256 \times 256 \times 3)$$

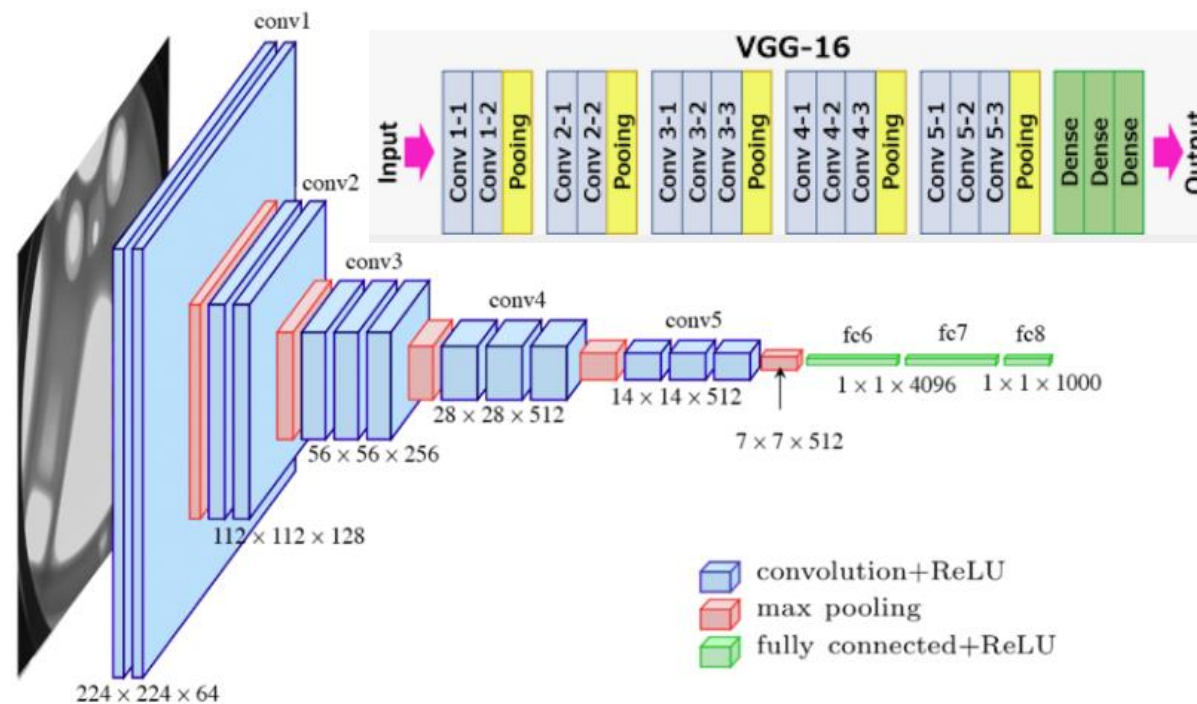
...

Redes neuronales para visión artificial

Variantes arquitecturales de clasificación populares (I)



- **VGGNet** (Simonian & Zisserman, 2014, [link](#)):
 - Comienzan a experimentarse ganancias simplemente aumentando la profundidad (y consecuentemente, el número de parámetros: **~138M** para VGGNet vs. ~62M AlexNet)...

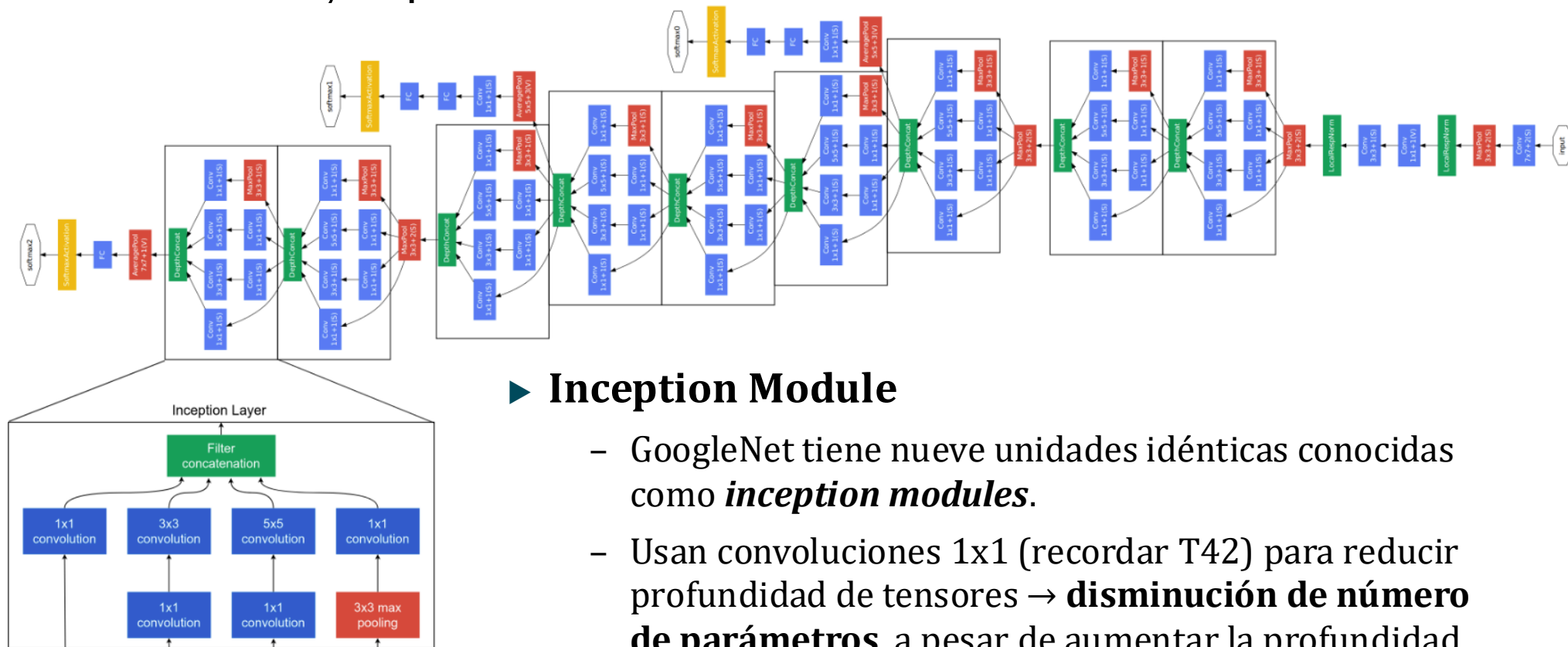


Redes neuronales para visión artificial

Variantes arquitecturales de clasificación populares (II)



- **GoogleNet** (Szegedy et al., 2014, [link](#)):
- Novedades arquitecturales buscando tanto mayor anchura como mayor profundidad en la CNN:

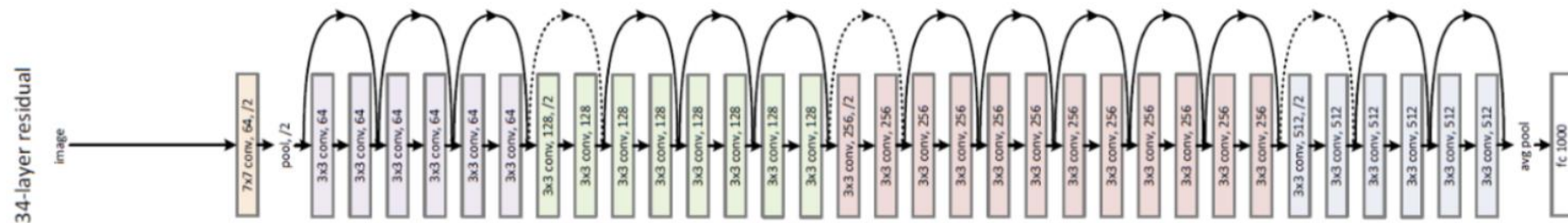


► Inception Module

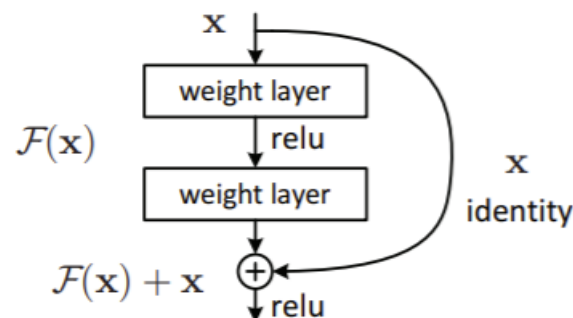
- GoogleNet tiene nueve unidades idénticas conocidas como *inception modules*.
- Usan convoluciones 1x1 (recordar T42) para reducir profundidad de tensores → **disminución de número de parámetros**, a pesar de aumentar la profundidad.
- AlexNet ~60M parámetros, GoogleNet ~7M parámetros.

► Bloques convolucionales residuales:

- Permiten flujos de gradientes más eficientes durante *training*.
- Posibilitan construcción de redes más profundas sin *overfitting* ni *gradient overflow/underflow*.



Bloque constructivo →
residual building block
([paper](#), 2015):



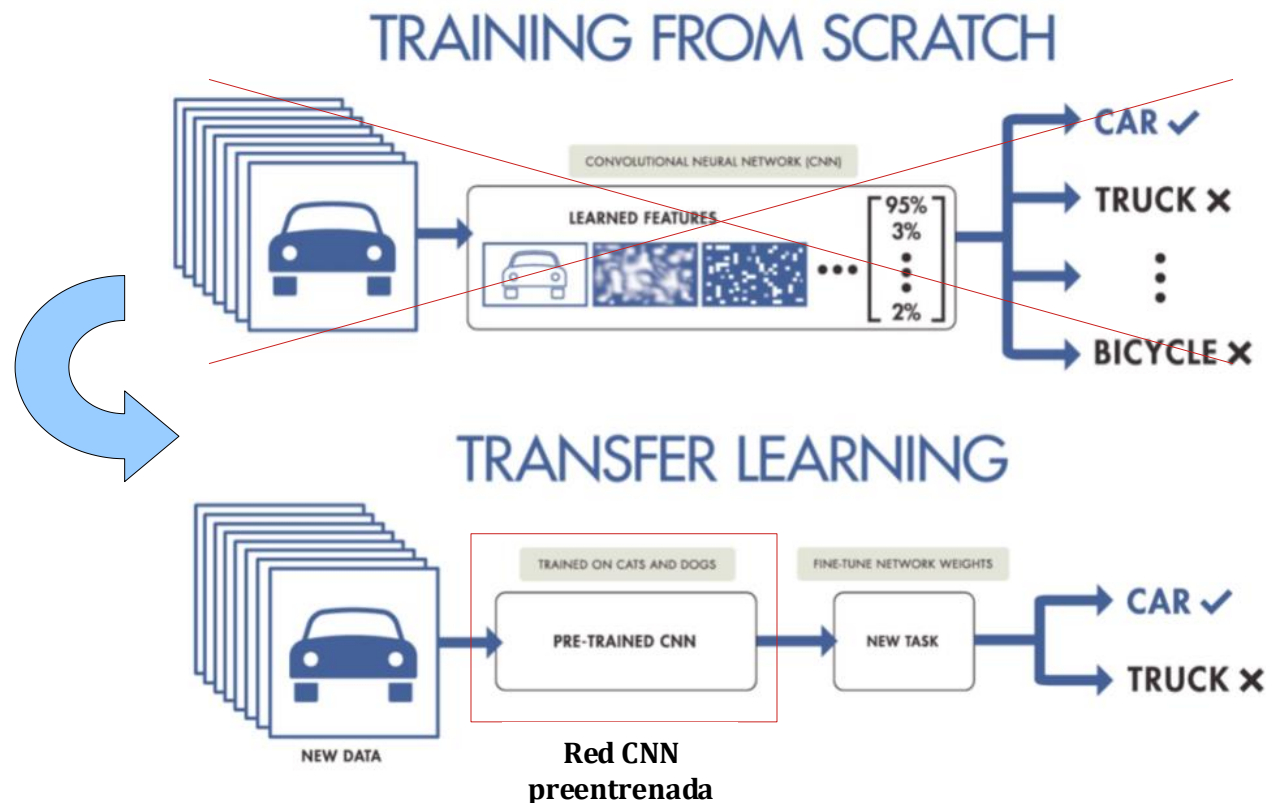
ResNet muy utilizado en la actualidad como “*backbones*” (base) de otras arquitecturas.

Redes neuronales para visión artificial

Transfer Learning



- Reutilizar el *backbone* (con pesos) de un modelo ya entrenado para una tarea diferente (con datos y clases nuevas):



Ejemplo completo en Google Colab ([link](#))

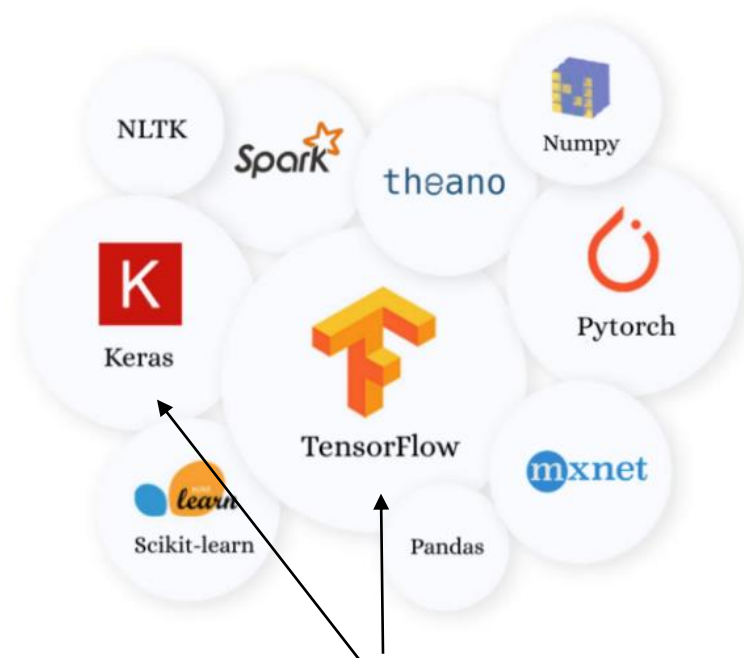
Redes neuronales para visión artificial

Software para Deep Learning



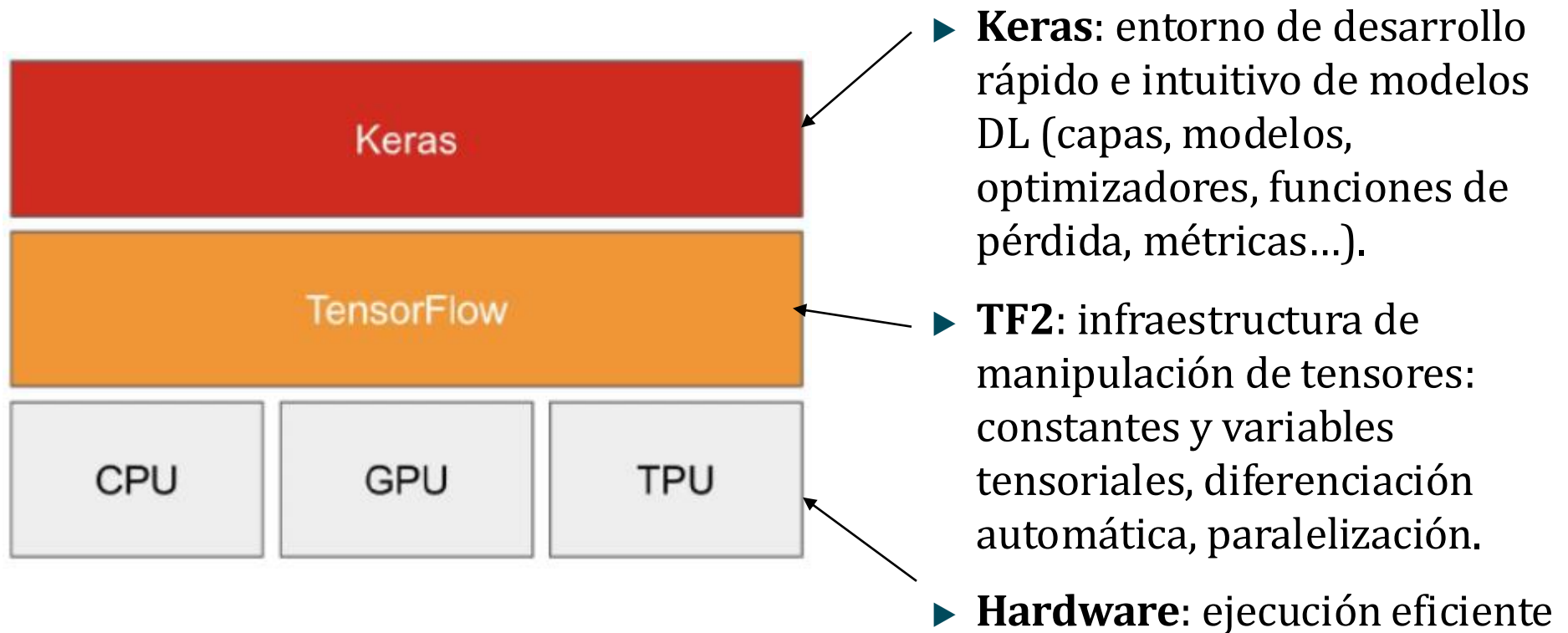
Máster en
Inteligencia
Artificial
UMU

- ▶ Gran variedad de librerías / *frameworks* para trabajar con DL:
 - No solo para visión por computador
 - También para otros ámbitos del *Machine Learning* (ML):
 - NLP
 - *Speech processing*
 - Detección de anomalías
 - Predicción de series temporales
 - etc.
- ▶ La gran mayoría es **software libre** sobre el ecosistema **Python**
- ▶ Las dos mas usadas en la actualidad son **Tensorflow** (Google; incluye **Keras**) & **Pytorch** (Meta)



Los notebooks de prácticas 8 y 9 usan fundamentalmente **Tensorflow 2 + Keras** (ya embebido en TF2)

► Relación entre Keras y Tensorflow:

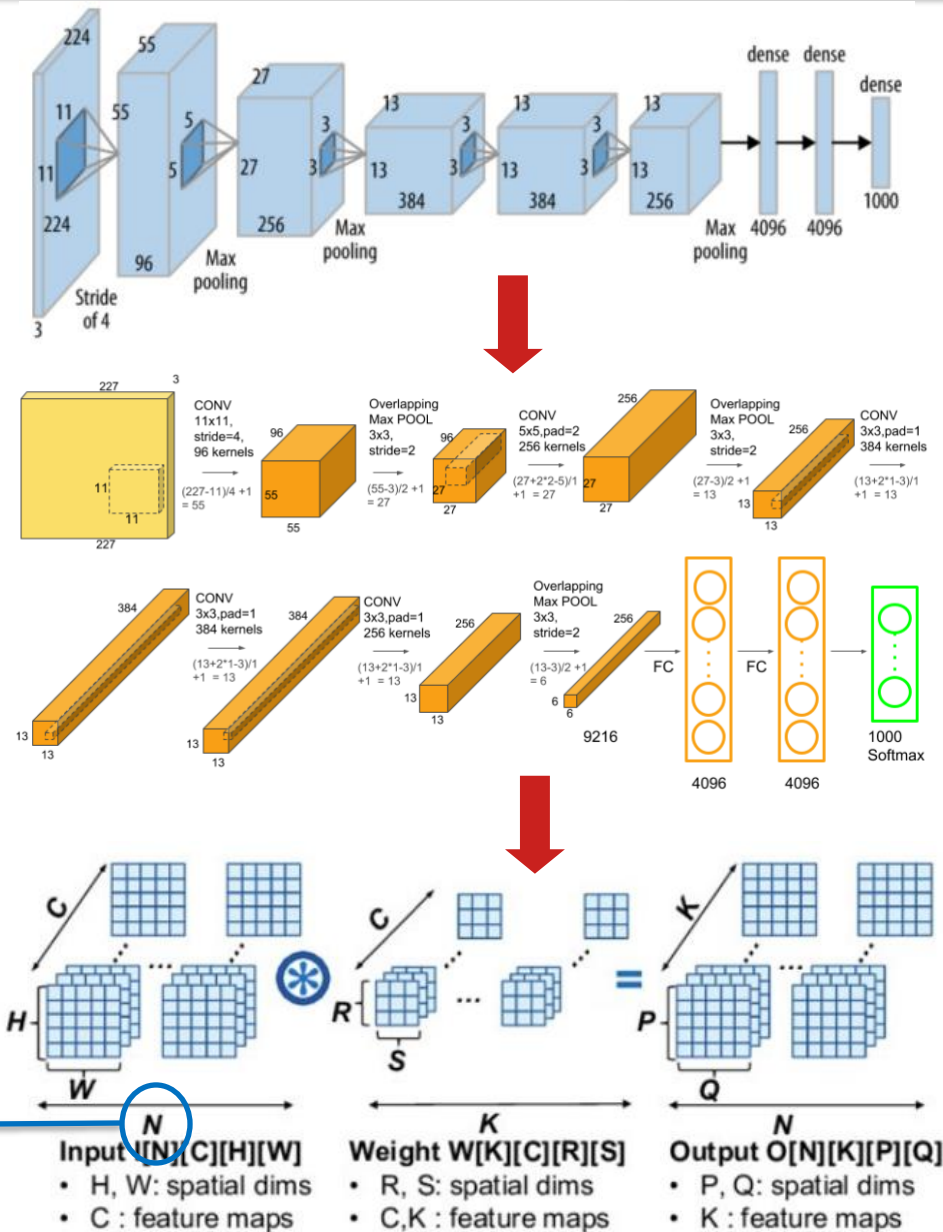


Redes neuronales para visión artificial

Estructura fundamental: tensores



- ▶ Entradas / salidas / parámetros (pesos)...
- ▶ Durante toda la computación **fluyen** grandes masas numéricas con estructura **tensorial**:
- ▶ Generalización a N dimensiones de los conceptos de vector (1D) y matriz (2D)



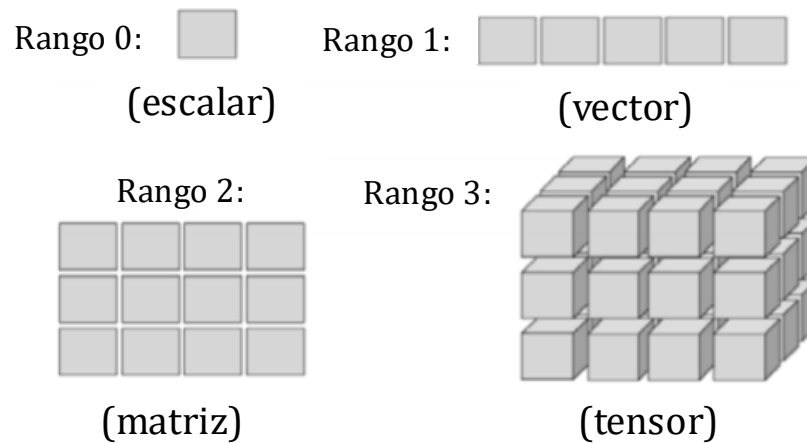
$N = \text{batch de imágenes}$

Redes neuronales para visión artificial

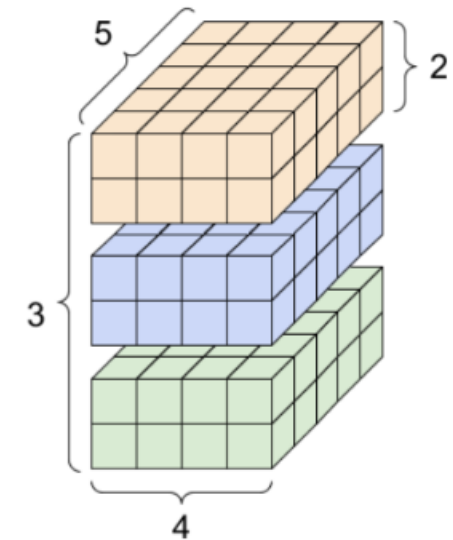
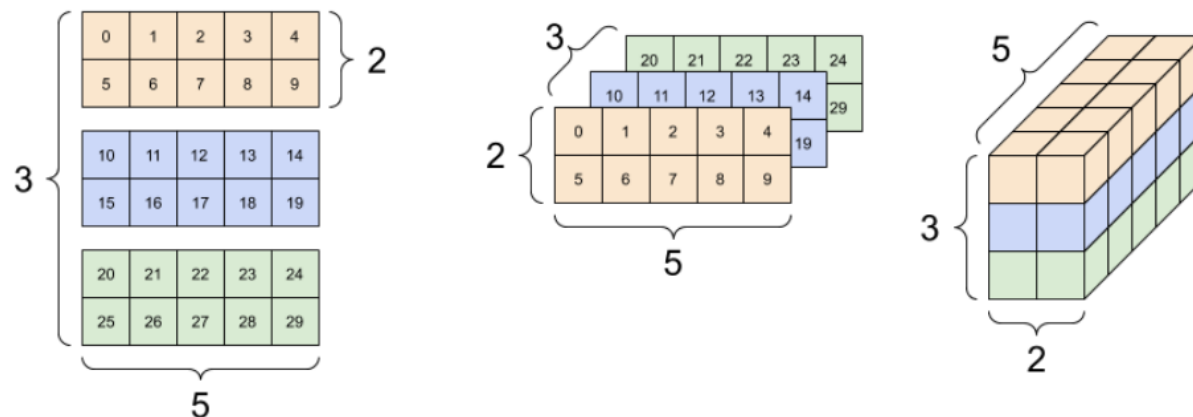
Tensores en TF2 (I)



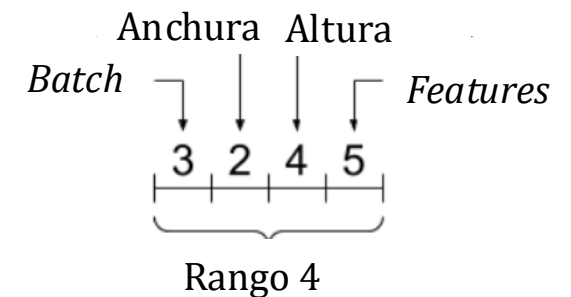
► **Numpy** es la librería subyacente → nos vale todo lo aprendido:



Ejemplo de un tensor con tres dimensiones [3,2,5]:



Orden típico de ejes para CNNs:

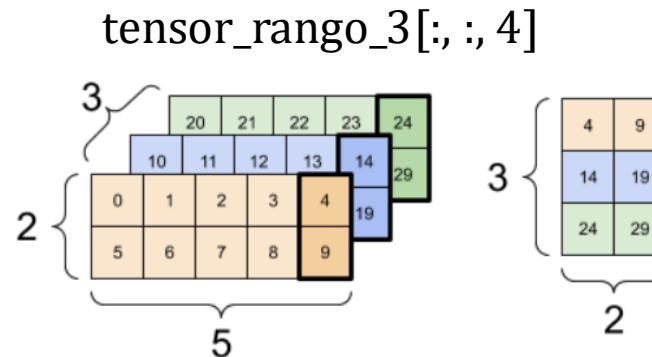


Redes neuronales para visión artificial

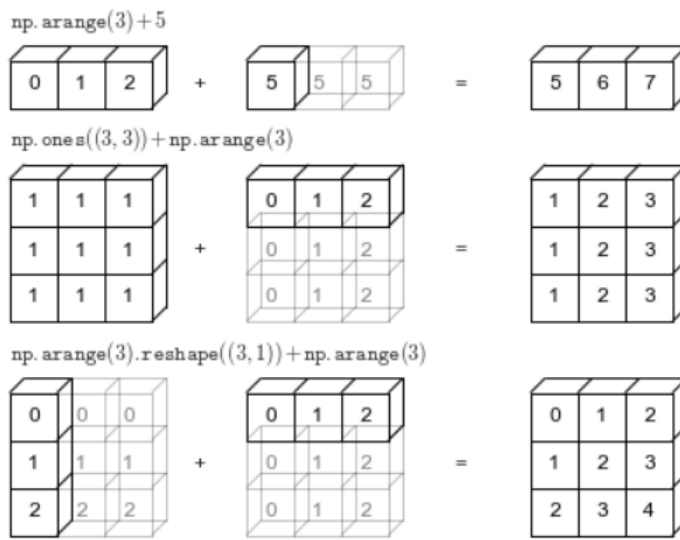
Tensores en TF2 (II)



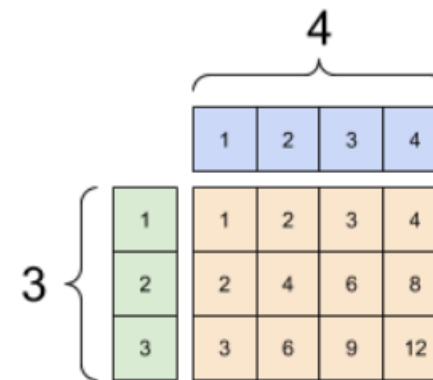
► *Slicing*:



► *Broadcasting*:



Ejemplo de *broadcasting* sobre una multiplicación de vectores:



Recordatorio *broadcasting* tensores “numpy style”: [link](#)

► Introducción a CNNs y algunos *datasets* básicos para clasificación (`VC_using_CNNs.ipynb`):

– ***MNIST & Fashion MNIST:***

- Ejemplos completos de ajuste y utilización CNNs en problemas de clasificación de imágenes:



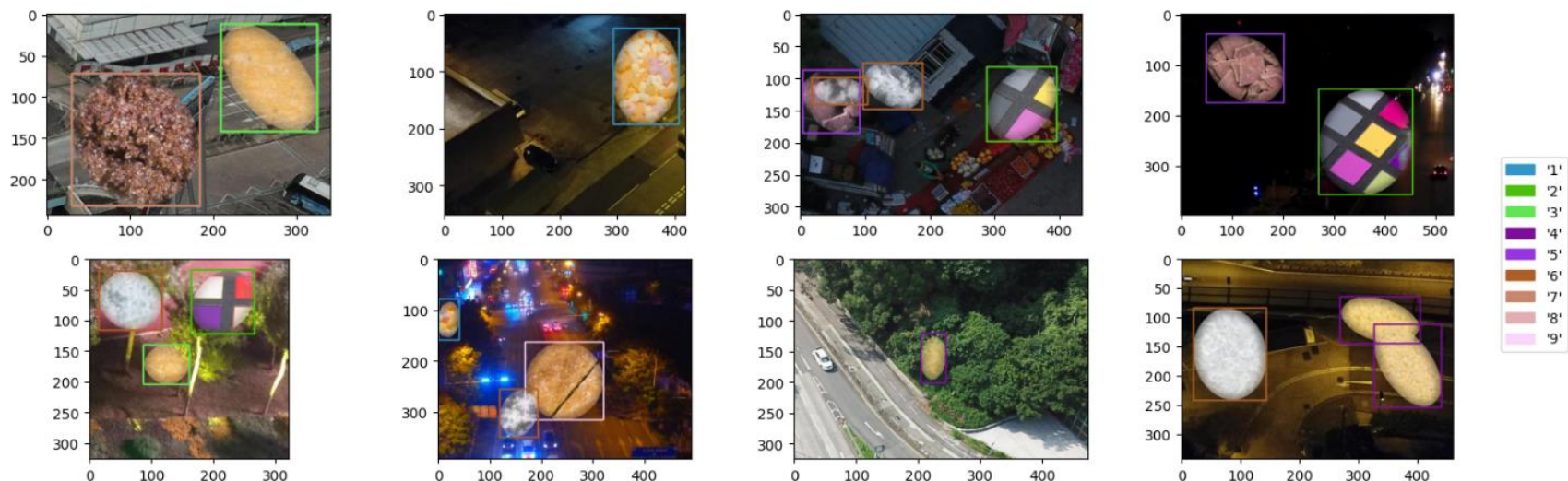
– Muy conocidos y utilizados para pruebas de concepto:

- Tamaño manejable (~60K ejemplos).
- Imágenes baja resolución.
- (→ carga computacional de *training* moderada).

- Comparación CNNs – *hand engineered features* (ALOTexturesCNNs.ipynb):
 - **ALOTextures** (clasificación/segmentación de texturas):



- *Toy-problem* asociado en detección de objetos (YOLO_OD_ALOTexturesCNNs.ipynb):



- ▶ Conceptos fundamentales en dichos *notebooks*:
 - Definición de modelos en Keras-TF2:
 - **API secuencial**
 - El más sencillo; pero menos potente que el funcional y el orientado a objetos.
 - **Ejemplos de capas en una CNN**
 - Convolucionales (y sus parámetros), *pooling*, *softmax*, ...
 - **Optimización**
 - Bucle de *training*
 - Introducción al *transfer learning*
 - Arquitecturas más allá de la clasificación:
 - Ejemplo de detección de objetos con librería [ultralytics](#).

Redes neuronales para visión artificial

Hoy: CNNs en todos los ámbitos de interés de la VA (I)



Máster en
Inteligencia
Artificial
UMU

Clasificación:



Recuperación de imágenes:



- Esqueleto (backbone) de la red muchas veces común...
 - ... pero, naturalmente, cambia la estructura de la salida (cabeza, o head) según el problema:
 - (Clasificación → vectores 0/1 ; localización objetos → ídem + coordenadas cajas ; segmentación → imagen de etiquetas; etc.)

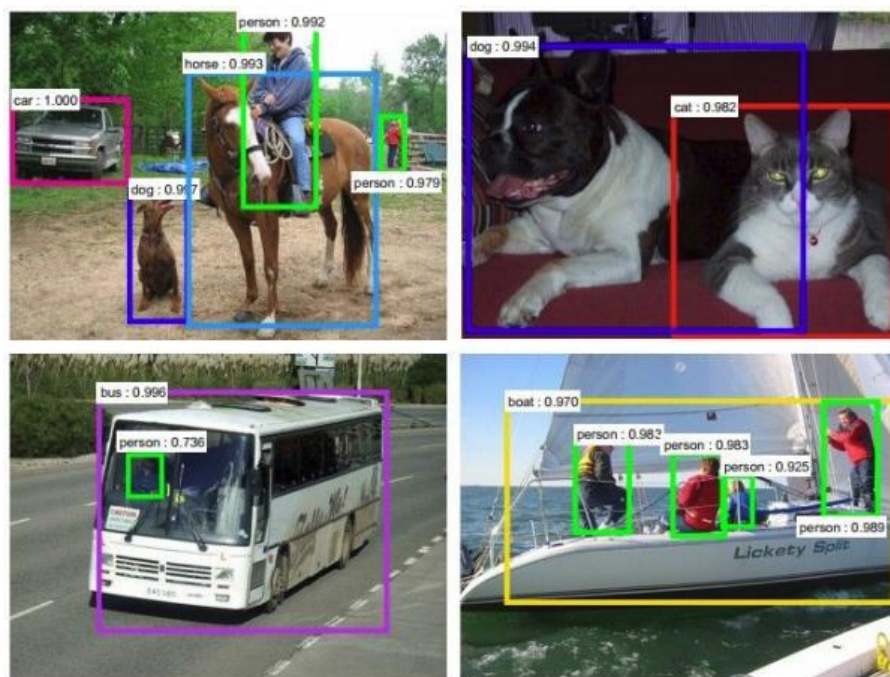
Redes neuronales para visión artificial

Hoy: CNNs en todos los ámbitos de interés de la VA (II)

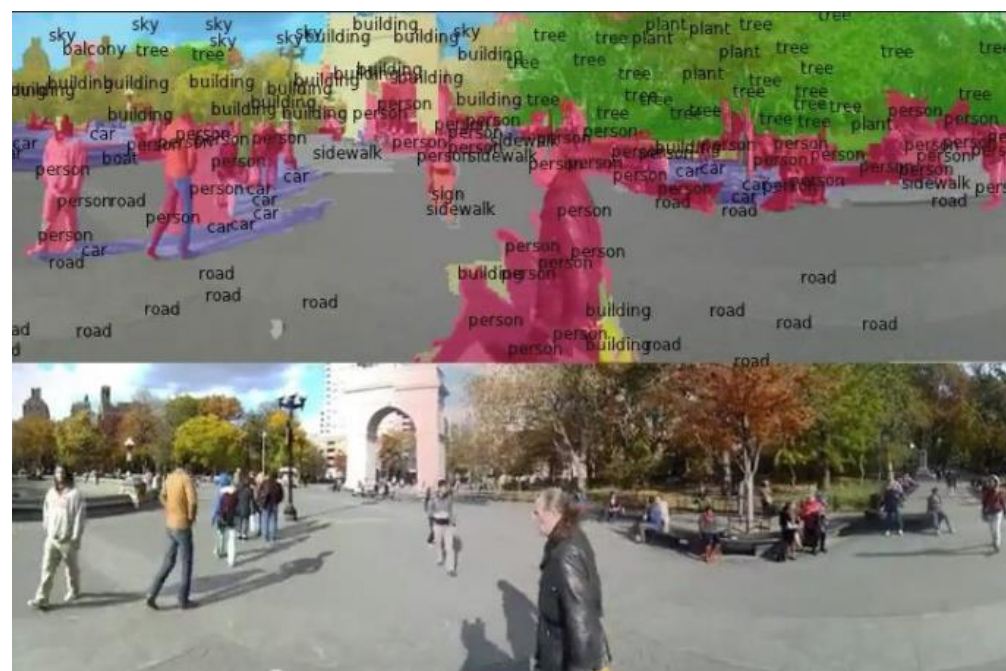


Máster en
Inteligencia
Artificial
UMU

Detección y localización de objetos:



Segmentación semántica / de instancias:



Segmentación semántica +
detección de objetos ([video](#))

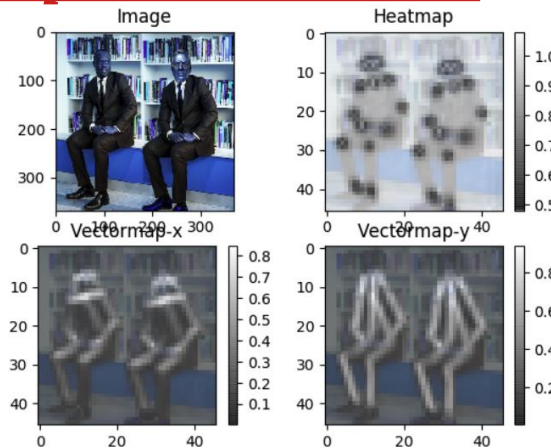
Redes neuronales para visión artificial

Hoy: CNNs en todos los ámbitos de interés de la VA (III)



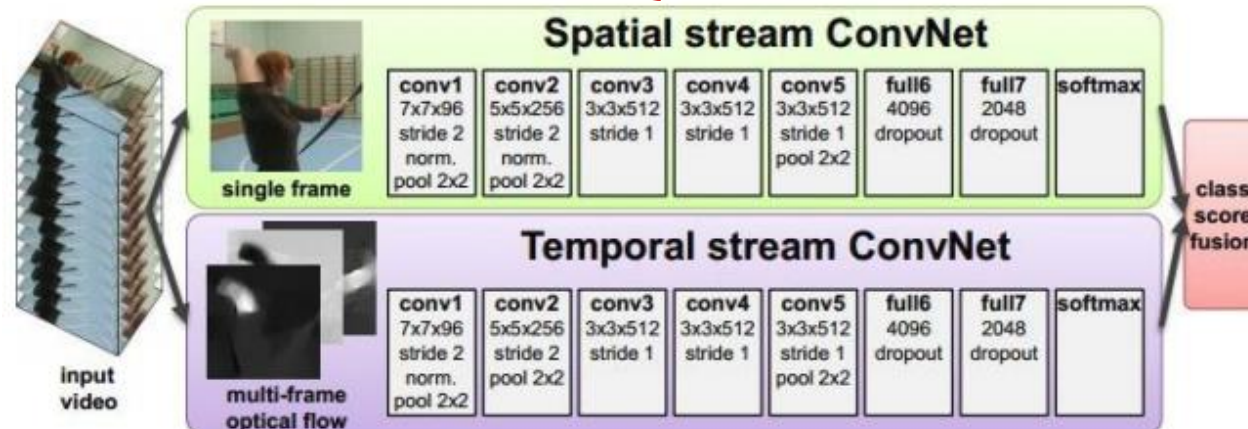
Máster en
Inteligencia
Artificial
UMU

Estimación de *pose* humana:



Ejemplo TensorFlow: [enlace](#)

Reconocimiento de actividad (tratamiento de vídeos):



Ejemplo [video](#)

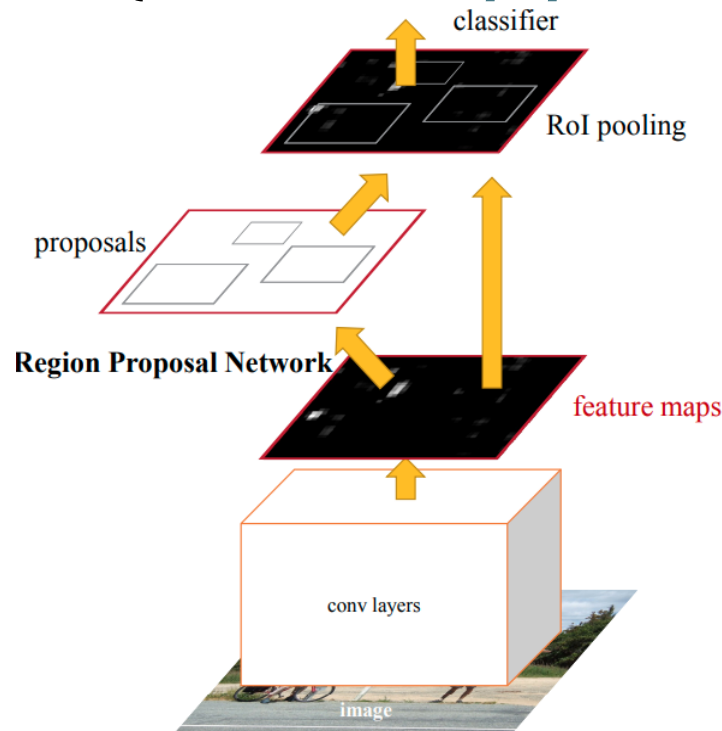
Redes neuronales para visión artificial

Arquitecturas convolucionales adaptadas a cada problema (I)

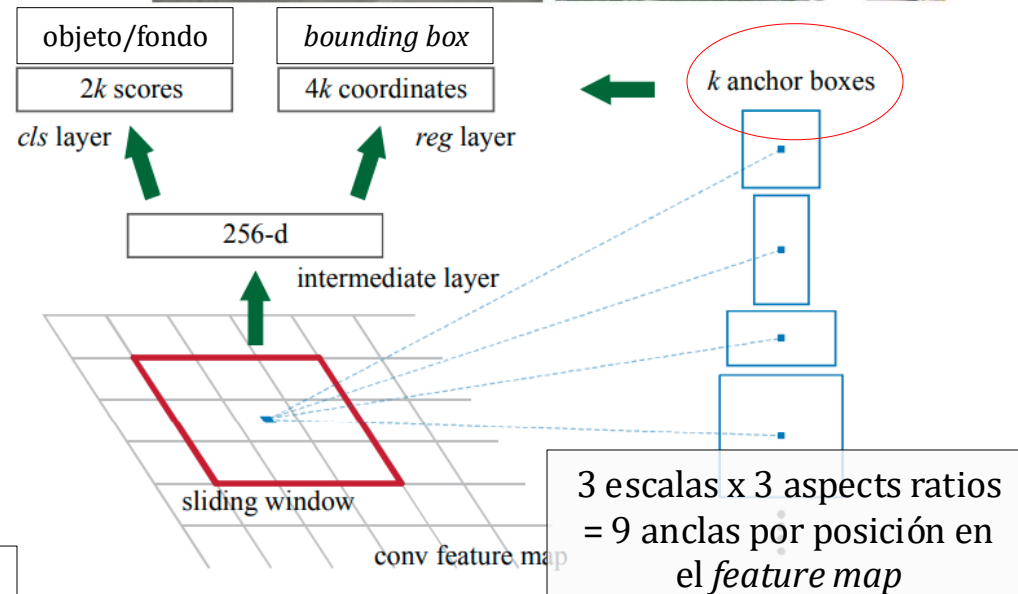
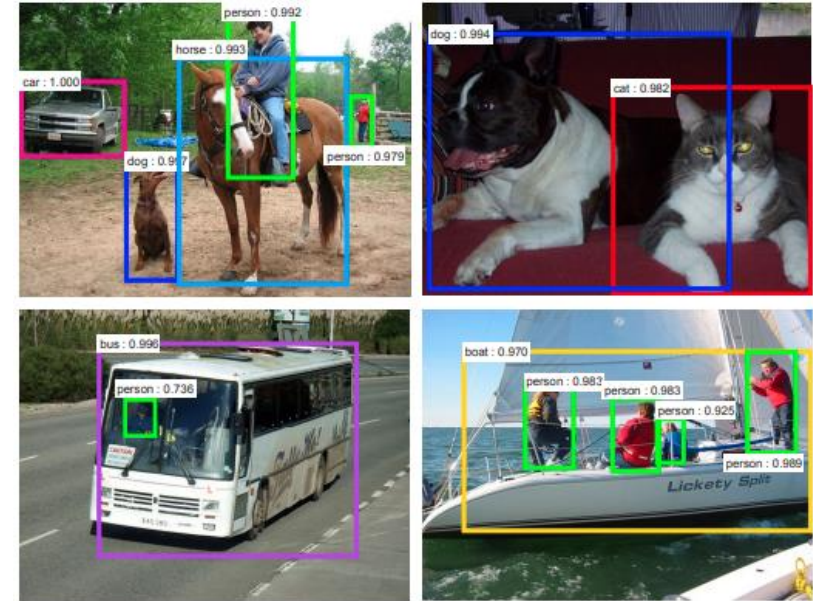


► 1. Detección de objetos (OD):

- Localización y clasificación simultánea de categorías.
- 1ª aprox: *Region proposal nets* (*Faster-RCNN*, [paper](#) 2015):



Detección en **dos pasos** (“two-stage OD”)

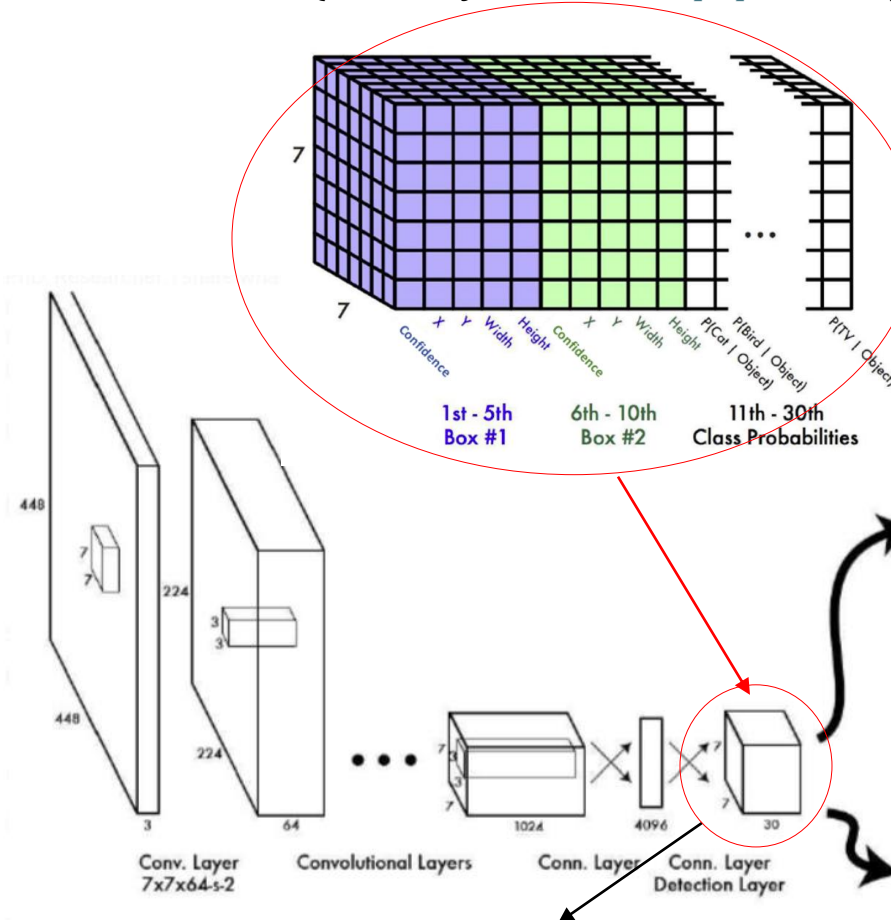


Redes neuronales para visión artificial

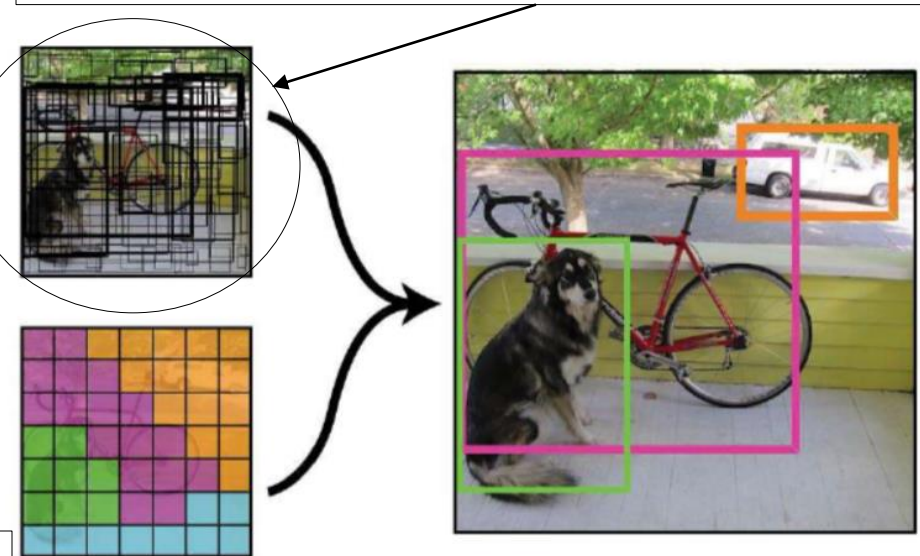
Arquitecturas para *Object Detection* (OD) (I)



- Eliminación de etapa “*region proposal*” (y *anchors*; ya sólo hasta dos cajas por celda):
 - YOLO V1 (“*You Only Look Once*”, [paper](#) 2016):

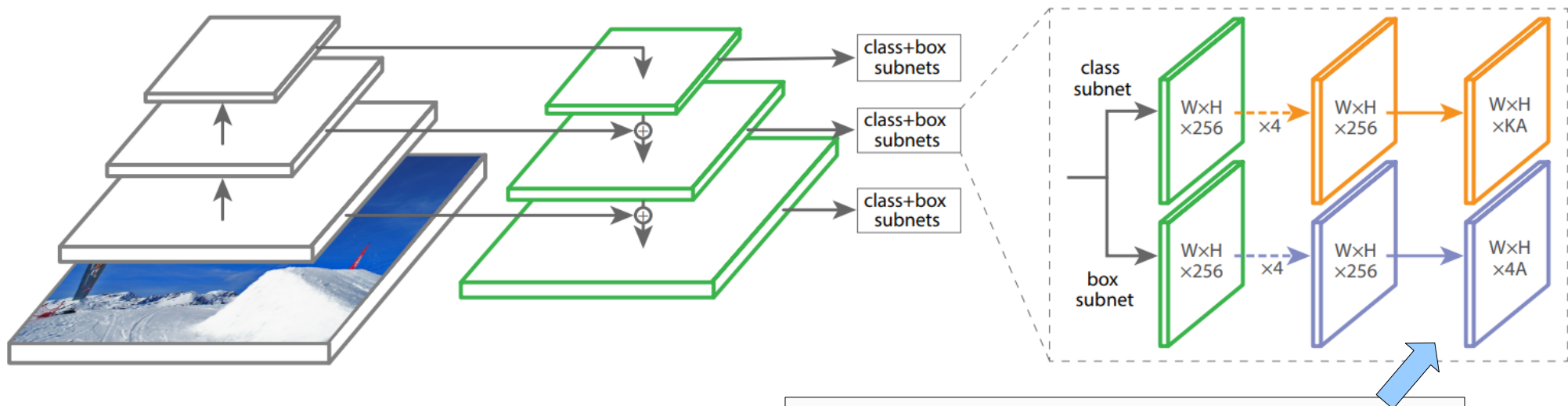


Las *bounding boxes* y confianzas obtenidas deben ser post-procesadas por un algoritmo de supresión de no máximos(NMS) para producir las coordenadas finales de las cajas y las correspondientes etiquetas de categoría.



Hasta 2 objetos (*bounding boxes*) por celda (con sus respectivas *confidences*), 20 clases

- **Tratamiento explícito multiescala (à la SIFT):**
 - **RetinaNet** ([paper](#), 2017).
 - *Backbone* (intercambiable) con **pirámide multiescala asociada**.
 - Cabezas (*heads*) regresión y clasificación reentrenables.



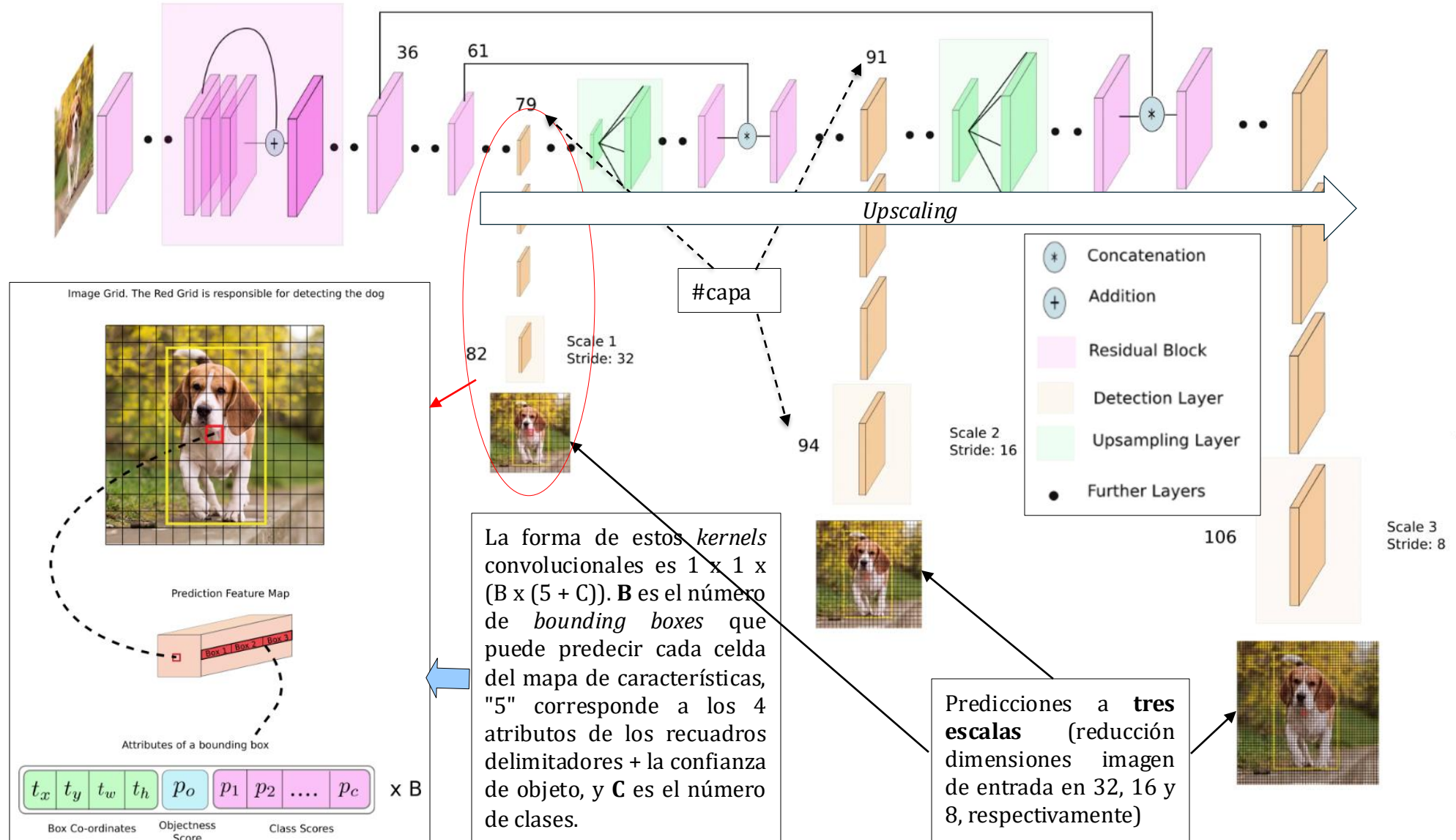
$K = \# \text{clases}$
 $A = \# \text{anclas}$
 $W \times H = \text{tamaño tensor salida (diferentes factores de reducción para diferentes escalas)}$

Redes neuronales para visión artificial

Arquitecturas para *Object Detection* (OD) (III)

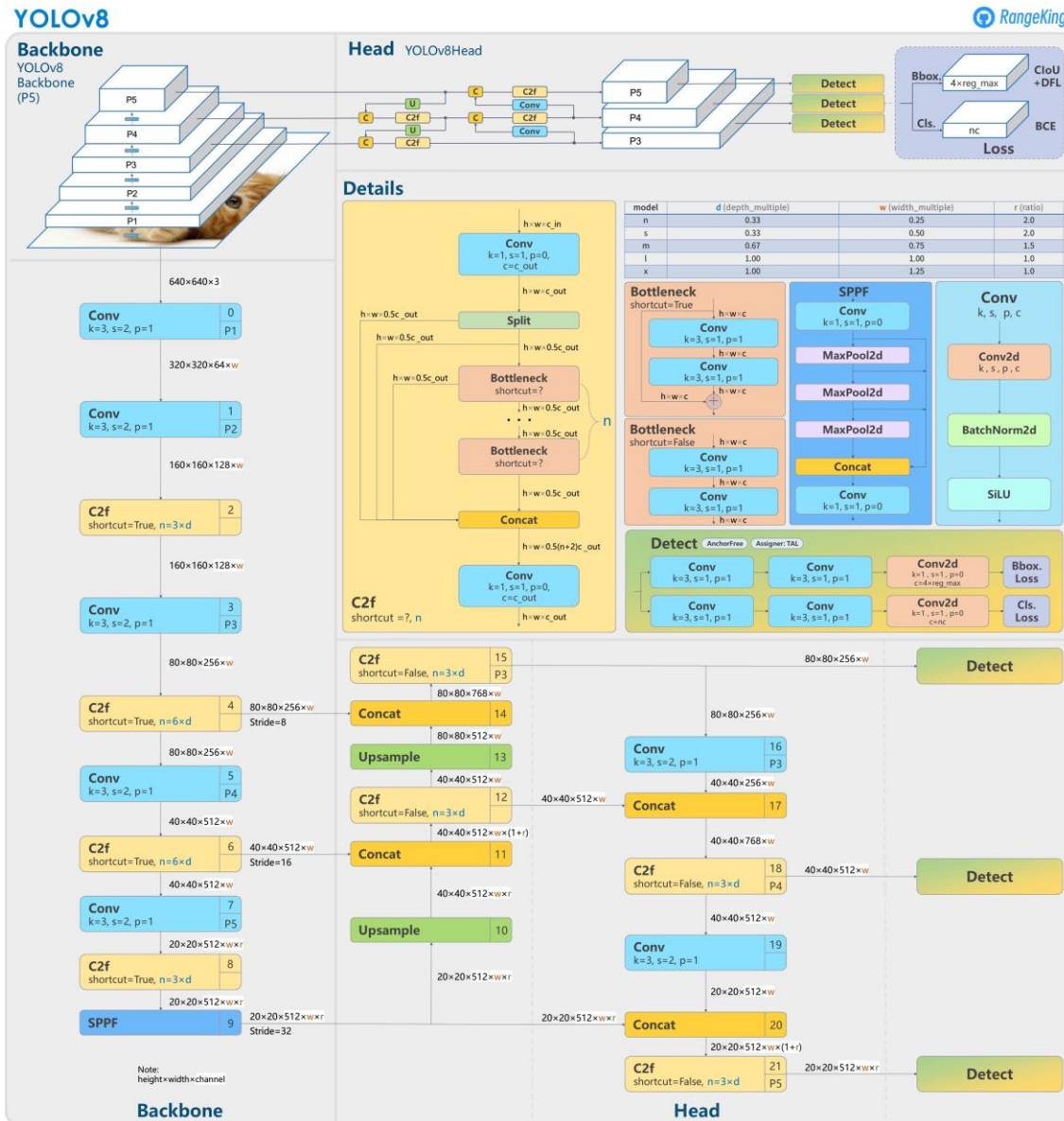


► Soluciones multiescala alternativas (YOLOV3 [paper](#), 2018):



Redes neuronales para visión artificial

Arquitecturas para Object Detection (OD) (IV)



- YOLOV8 y posteriores (2023-):

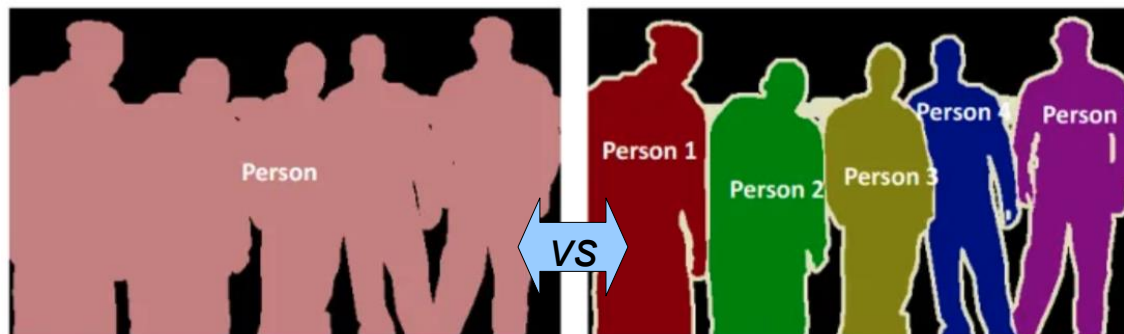
- Filosofía similar a RetinaNet y YOLOs anteriores (one-stage, multiescala, etc.), pero...
- ... estructura red *backbone* más compleja.
- Incluso autoaprendida (***Network Architectural Search***: [YOLO-NAS](#)).
- Transfer learning casi trivial → lo veremos en prácticas
(YOLO_OD_ALOTextures.ipynb)
- [Ultralytics notebooks](#).

► 2. Segmentación:

- Clasificación pixel a pixel en toda la imagen:



- *Image segmentation* vs. *Instance segmentation*:

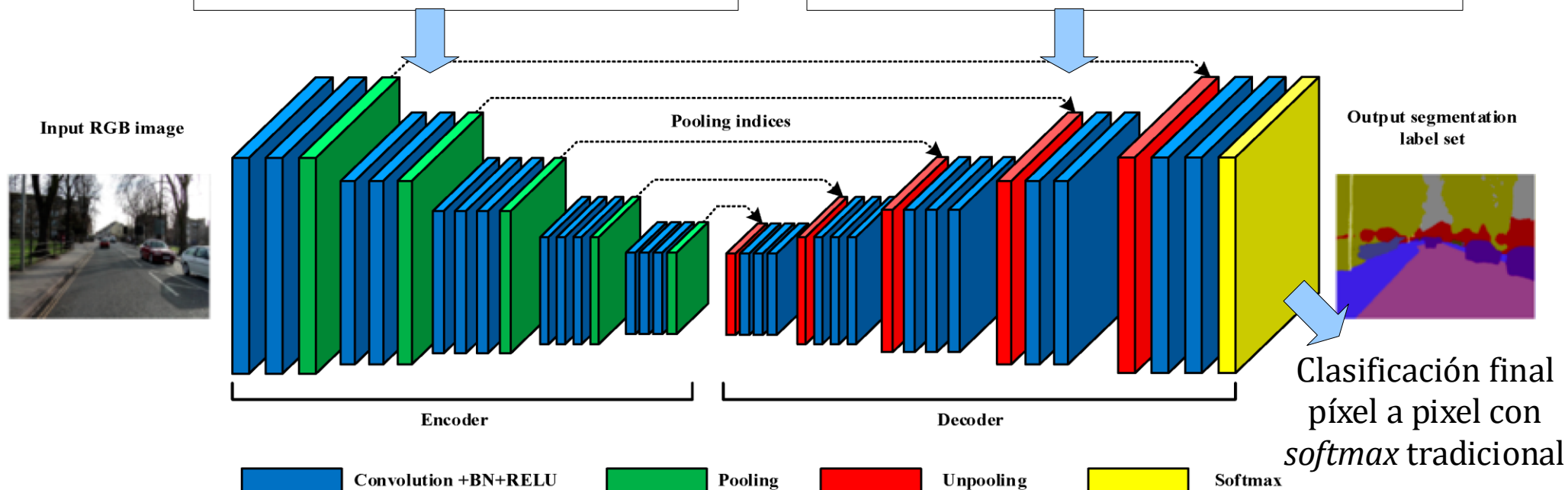


► Arquitectura genérica **encoder-decoder**:

- Idea muy utilizada en DL en general (no sólo visión):

Encoder: extrae características cada vez más semánticas (mayor profundidad tensores, a menor resolución, progresivamente)

Decoder: recupera resolución, mediante *upsampling (convoluciones transpuestas)* + combinación con mapas de características del *encoder* correspondientes a cada escala.

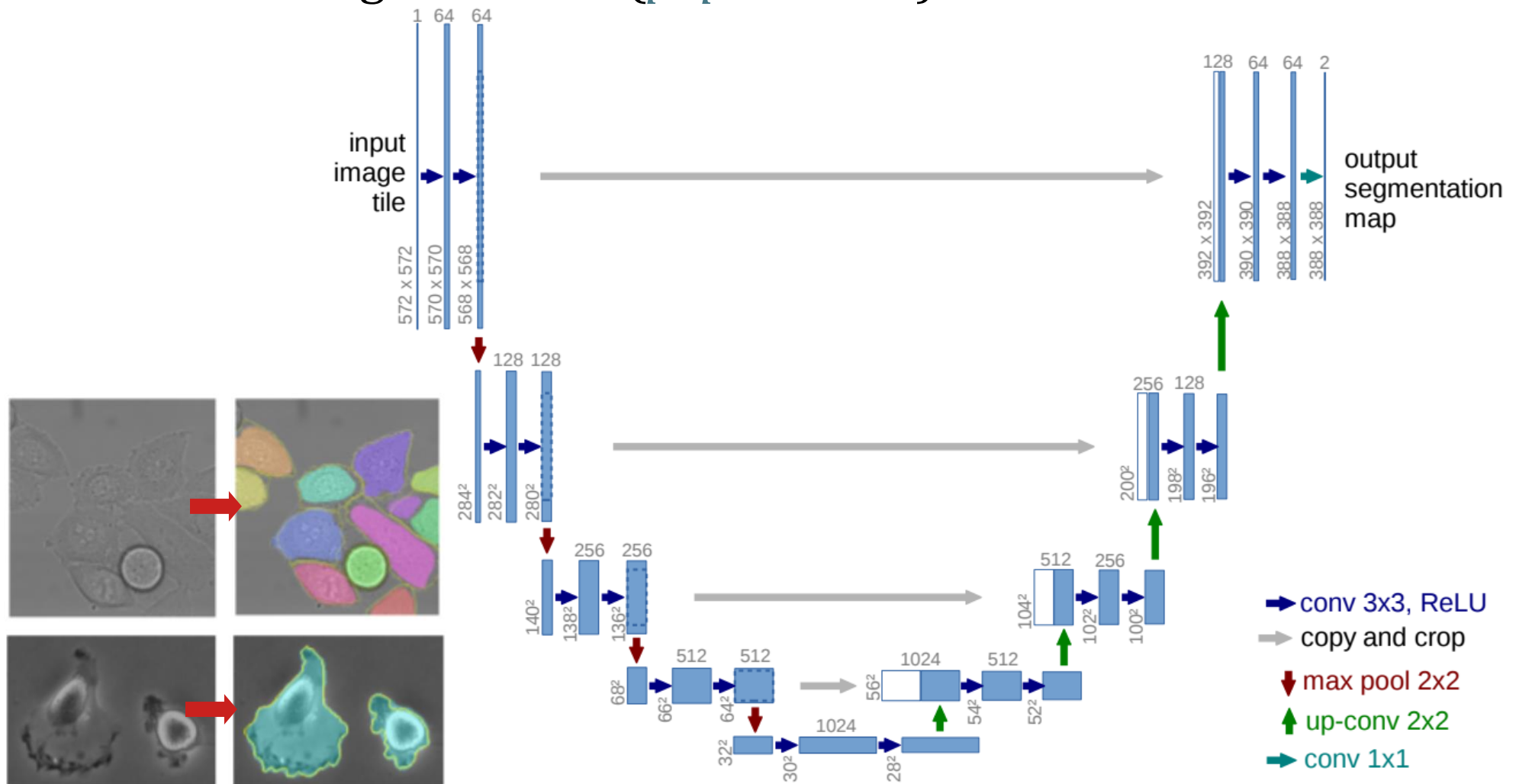


Redes neuronales para visión artificial

Arquitectura común segmentación (II)



- **U-Net:** 1ª arquitectura seminal segmentación píxel a píxel, ámbito imagen médica ([paper](#), 2015):



Redes neuronales para visión artificial

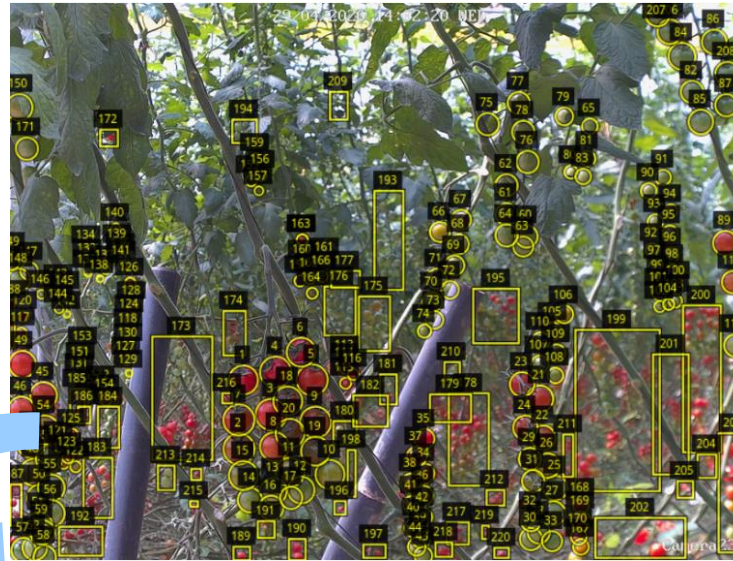
Arquitectura común segmentación (III)



Máster en
Inteligencia
Artificial
UMU

► Ejemplo uso U-Nets en agricultura inteligente (UMU 2021):

Training:

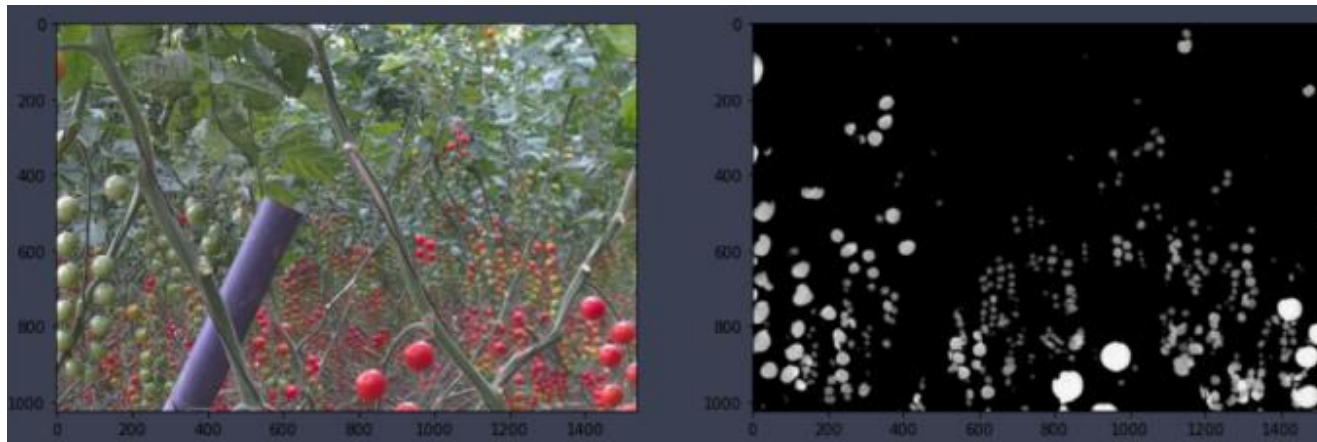


Segmentación, conteo,
medida y extracción
de propiedades de frutos
en explotaciones agrícolas

Etiquetado *training*:

- Círculos → positivos
- Rectángulos → ignorar

Test:



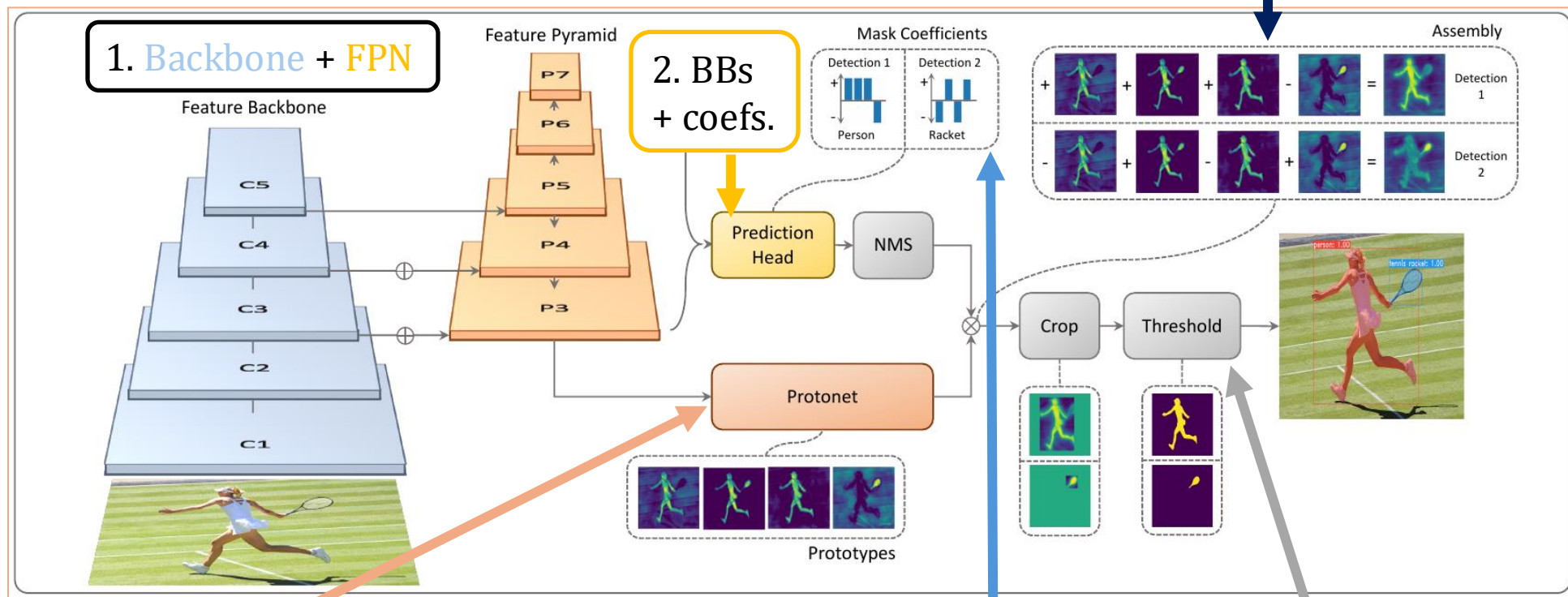
Redes neuronales para visión artificial

Segmentación de instancias



- Combinación OD + máscaras de segmentación
 - YOLACT (You only look at coefficients) [paper](#) (2019).
 - Idea heredada en YOLOv11-segm (2024).

6. Instancias resultado
(una por *bounding box*)



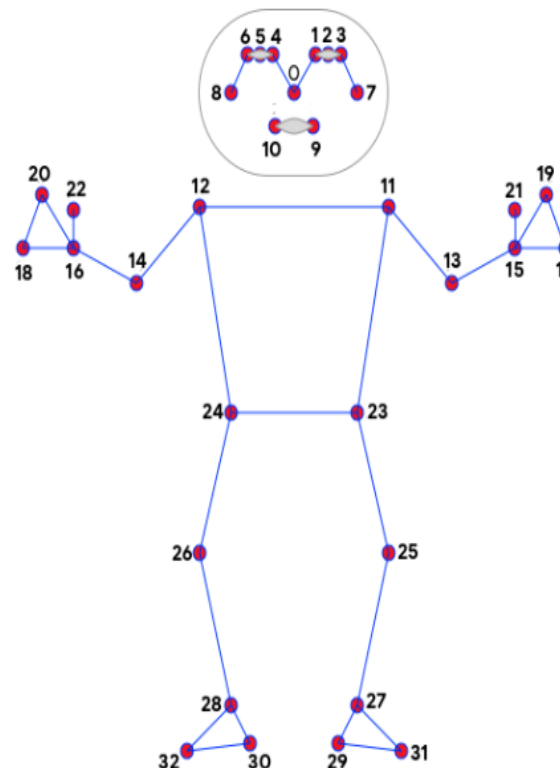
3. Cabeza de generación de "base vectorial" de máscaras (prototipos) → entrenable

4. Coeficientes lineales para combinar máscaras de la base → entrenables

5. Módulos NMS (*bounding box*), *cropping*, *threshold* → no entrenables

► 3. Estimación de *pose*:

- Detección de puntos de interés en objetos (*pose* humana lo más común)
- Una clase por tipo de punto de interés (articulaciones humanas → *joints*)



Landmarks

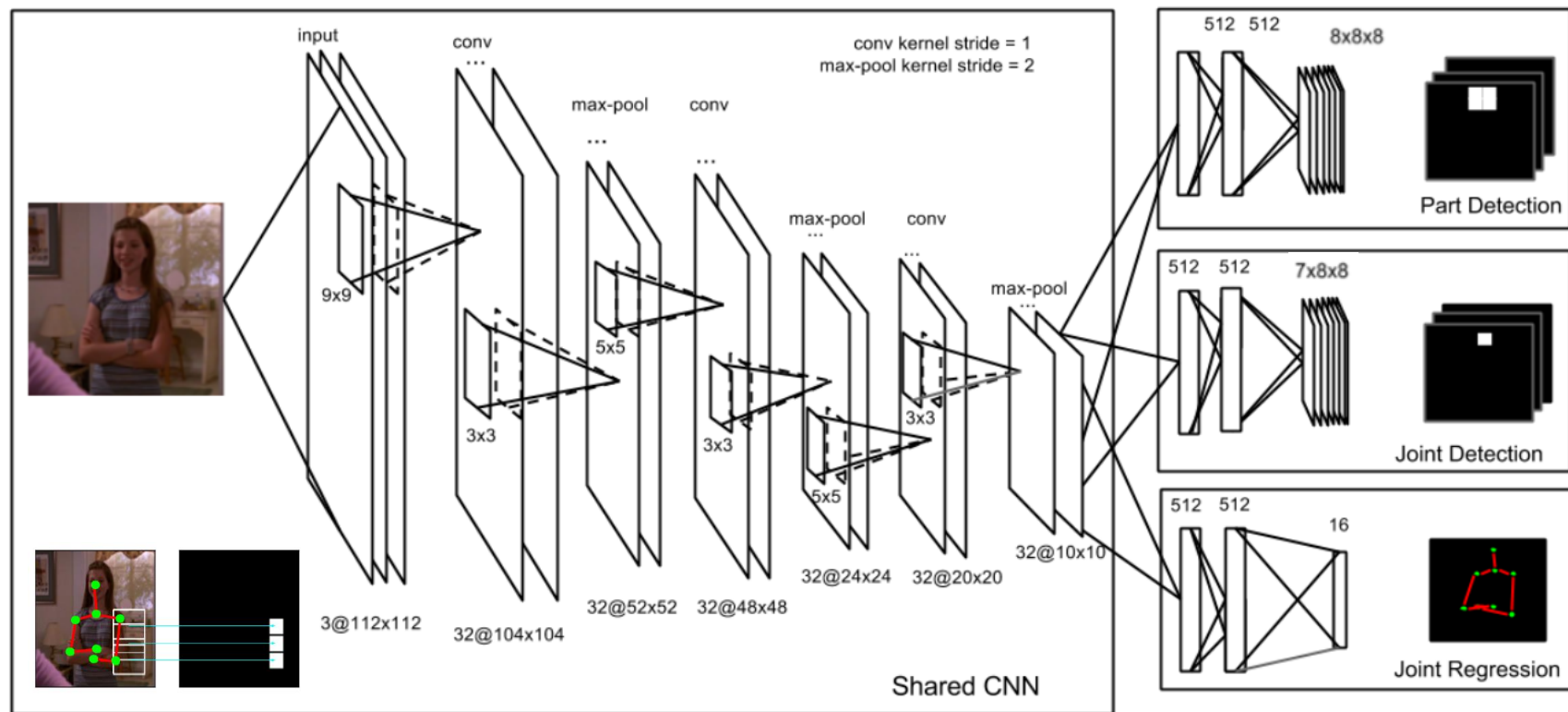
- | | |
|--------------------|----------------------|
| 0. Nose | 17. Left_pinky |
| 1. Left_eye_inner | 18. Right_pinky |
| 2. Left_eye | 19. Left_index |
| 3. Left_eye_outer | 20. Right_index |
| 4. Right_eye_inner | 21. Left_thumb |
| 5. Right_eye | 22. Right_thumb |
| 6. Right_eye_outer | 23. Left_hip |
| 7. Left_ear | 24. Right_hip |
| 8. Right_ear | 25. Left_knee |
| 9. Left_mouth | 26. Right_knee |
| 10. Right_mouth | 27. Left_ankle |
| 11. Left_shoulder | 28. Right_ankle |
| 12. Right_shoulder | 29. Left_heel |
| 13. Left_elbow | 30. Right_heel |
| 14. Right_elbow | 31. Left_foot_index |
| 15. Left_wrist | 32. Right_foot_index |
| 16. Right_wrist | |

Redes neuronales para visión artificial

Arquitectura básica para estimación de *pose* (I)



- *Backbone* CNN clásico + *upsampling* final y/o regresión fina:
 - Una clase por tipo de punto de interés (**#joints** → **profundidad tensor salida**).
 - (En este artículo concreto ([link](#)), también tensor de salida para partes y para posiciones exactas de regresión):

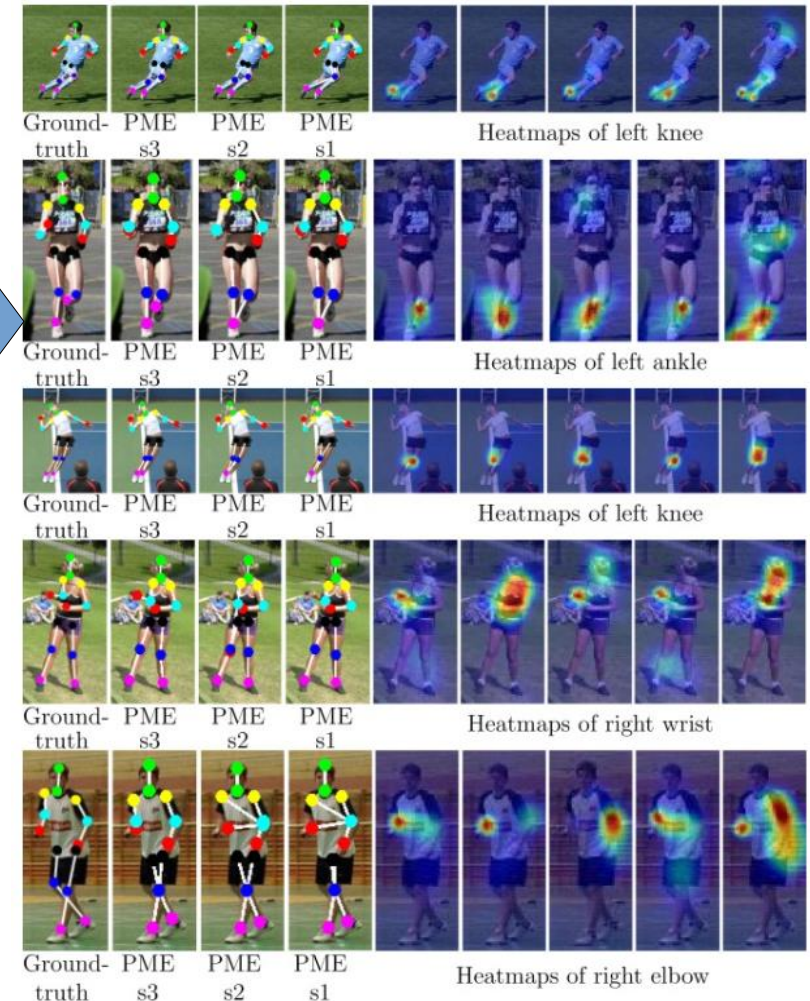
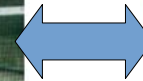
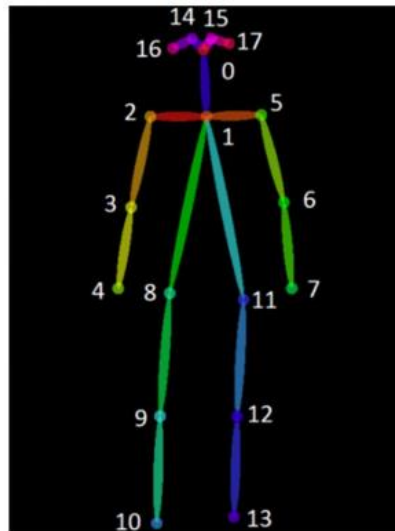


Redes neuronales para visión artificial

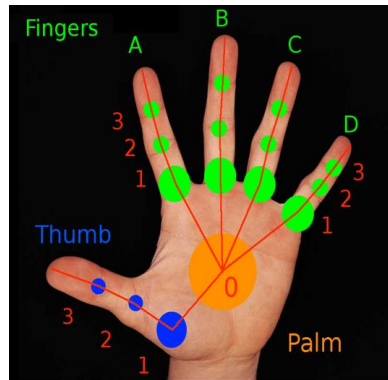
Arquitectura básica para estimación de pose (II)



- Si el enfoque incluye *upsampling*, cada slice del tensor de salida → puede interpretarse como un heatmap:



- Modelo pose mano análogo:



- ▶ Curso “*Introduction to Deep Learning*”, MIT, 2022.
<http://introtodeeplearning.com/>
- ▶ Curso “*Convolutional Neural Networks for Visual Recognition*”, Univ. Stanford, 2024.
<http://cs231n.stanford.edu/>
- ▶ “*Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: concepts, tools and techniques to build intelligent systems*”, 3rd ed. Aurélien Géron, O'Reilly, 2022.
<https://www.oreilly.com/library/view/hands-on-machine-learning/9781098125967/>
- ▶ “*Deep Learning*”, I. Goodfellow, I. Bengio, A. Courville, MIT Press 2016.
<http://www.deeplearningbook.org/>
- ▶ “*Neural Networks and Deep Learning*”, M. Nielsen, Online book, 2017.
<http://neuralnetworksanddeeplearning.com/>
- ▶ “*Tensorflow: Guía y tutoriales*” (incluye Keras), Google, Online doc., 2023.
<https://www.tensorflow.org/overview/>
- ▶ “*Deep Learning, Foundations and Concepts*”, C.M. Bishop, H. Bishop, Springer, 2024.
<https://link.springer.com/book/10.1007/978-3-031-45468-4>